

Diplomarbeit im Studiengang
Angewandte Informatik
an der Universität Augsburg

Portierung der XML-Abfragesprache Preference XPath auf Java 1.5

Michael Schierl

Mai 2006

Erstgutachter: Prof. Dr. Werner Kießling
Zweitgutachter: Prof. Dr. Bernhard Möller
Betreuer: Sven Döring

Inhaltsverzeichnis

1	Einleitung	1
1.1	Motivation	1
1.2	Überblick	2
2	Grundlagen	4
2.1	Präferenzen	4
2.1.1	Basispräferenzen	5
2.1.2	Basis-SV-Relationen	7
2.1.3	Zusammengesetzte Präferenzen	8
2.2	XPath	10
2.3	Das Preference-Framework	12
2.3.1	Die Basisklassen	12
2.3.2	Preference XPath	14
3	Erweiterung der Basisklassen	21
3.1	Fehlerbehebung	21
3.2	Vorbereitungen	22
3.3	Serialisieren/Deserialisieren	24
3.4	LAYERED-Präferenz	25
3.5	D-Parameter und SV-Relationen	26
3.6	Erweiterungsklassen	28
4	Anpassung an JDK1.5	29
4.1	Probleme und Lösungsmöglichkeiten	29
4.2	Die Adapterschnittstelle	31
4.3	Die Fassadenklassen	35
4.4	Selbstdefinierte XPath-Funktionen	39
4.5	Implementationen der Adapterschnittstelle	41
4.5.1	Adapter für JDK1.4.1/1.4.2	41
4.5.2	Die saubere Lösung	43
4.5.3	Steigerung der Performance unter JDK1.5.0	44
4.5.4	Performanceoptimierte Lösung nur für JDK1.5.0	45
4.5.5	Verwendung externer XML-Parser (Apache Xerces/Xalan)	45

4.5.6	Vergleich der Implementationen	45
4.6	Buildfile	46
5	Performance-Analysen	47
5.1	Verwendete Datenbanken	47
5.2	Testumgebung	48
5.3	Lastverteilung	49
5.4	Testergebnisse	50
5.4.1	Initialisierungszeiten	50
5.4.2	Anfragen an kleine Datenbanken	50
5.4.3	Anfragen an große Datenbanken	52
5.4.4	Abhängigkeit der Laufzeit von der Datenbankgröße	56
6	Zusammenfassung und Ausblick	58
6.1	Zusammenfassung	58
6.2	Ausblick	59
A	Inhalt der CD	I
B	Weitere Änderungen am Programmcode	II
B.1	Kleinigkeiten	II
B.2	Anpassungen des Parsers	III
B.3	Testframework, Beispiel-GUI	IV
B.4	Xindice-Anbindung	VII
B.5	API-Dokumentation	VII
C	Rohdaten der Performance-Analysen	VIII

1 Einleitung

1.1 Motivation

Präferenzen sind ein wichtiger Bestandteil unseres Alltags – auch wenn wir uns dessen meist nicht bewusst sind. Formulierungen wie „Ich mag A lieber als B“ oder „Alles ist besser als X“ sind doch jedem geläufig.

Umso verwunderlicher ist es, dass die gängigen E-Business-Plattformen immer noch lediglich harte Suchkriterien anbieten. Dies führt dann dazu, dass man regelmäßig zunächst viel zu genaue Kriterien vorgibt, und daher keine Treffer erhält (der so genannte „empty result set effect“). Daraufhin entfernt man einige Kriterien oder wandelt ein Und-Kriterium in ein Oder-Kriterium um, worauf sich das Blatt wendet und man von unzähligen – hauptsächlich irrelevanten – Ergebnissen überschüttet wird (der so genannte „flooding effect“).

Die Verwendung von präferenzbasierten Suchen würde, sofern die Produkte bereits sinnvoll klassifiziert sind¹, diese Effekte vermeiden helfen und damit die Kundenzufriedenheit erhöhen.

XML ist ein textbasiertes Datenformat, das insbesondere im E-Commerce immer häufiger für Präsentation, Speicherung oder Austausch von strukturierten Daten verwendet wird. Standardisierte Formate wie *BMEcat* und *eCl@ss* vereinfachen den Datenaustausch zwischen Unternehmen. Die Abfragesprache XPath stellt eine bequeme Möglichkeit dar, Daten nach harten Kriterien aus einer XML-basierten Datenbank zu selektieren – ähnlich wie es SQL für relationale Datenbanken erlaubt.

Die präferenzbasierten Abfragesprachen Preference SQL und Preference XPath, die vom Lehrstuhl für Datenbank- und Informationssysteme der Universität Augsburg entwickelt wurden, bauen auf SQL bzw. XPath auf. Diese können auch an E-Business-Plattformen (wie COSIMA, vgl. [KFD04]) angebunden und von dort benutzt werden.

¹auch das ist in vielen Onlineshops leider nicht der Fall

Preference XPath wurde für Java 1.4.1 entwickelt und war deshalb auch nur unter dieser Version lauffähig. Allerdings ist die Herstellerunterstützung für Java 1.4.1 inzwischen ausgelaufen, wodurch die meisten Nutzer inzwischen auf Java 1.4.2 oder Java 1.5 (auch bekannt als J2SE 5.0) wechselten. Die zusätzlichen Klassenbibliotheken und Sprachfeatures von Java 1.5 haben die Upgradebereitschaft im Desktopbereich beschleunigt, so dass dort inzwischen überwiegend Java 1.5 zum Einsatz kommt. Im Serverbereich wird noch häufig Java 1.4.2 eingesetzt; dies wird in den kommenden Jahren aber weiter stagnieren.

Es ist zwar theoretisch möglich, mehrere Java-Laufzeitumgebungen auf einem System zu installieren; dies ist aber mit zusätzlichem Aufwand und Speicherplatzbedarf verbunden. Nicht ohne weiteres möglich ist allerdings die Verwendung von verschiedenen Java-Laufzeitumgebungen für einzelne Bibliotheken. Ein Programm, das Java-1.5-Sprachmerkmale benutzt, kann also keine Bibliothek (wie Preference XPath) direkt benutzen, die nur unter Java 1.4 läuft. Dieses Problem ließe sich theoretisch durch gleichzeitige Ausführung zweier Laufzeitumgebungen, die über Interprozesskommunikation (RMI) Daten austauschen, lösen. Dies führt aber zu erheblich höherem Speicher- und Rechenzeitverbrauch, weshalb es in der Praxis keine Alternative darstellt. Deshalb ist es für die Akzeptanz von Preference XPath hinderlich, wenn es ein bestimmtes (noch dazu inzwischen nicht mehr unterstütztes) JRE benötigt.

Aus diesem Grunde wurde das Preference-XPath-Framework im Rahmen dieser Diplomarbeit dahingehend angepasst, dass es mit Java 1.5.0 und zukünftigen Versionen einwandfrei funktioniert. Nachdem in der Praxis (insbesondere im Serverbereich) häufig noch Java 1.4.2 eingesetzt wird, soll das Programm aber weiterhin unter Java 1.4.2 lauffähig bleiben.

1.2 Überblick

Kapitel 2 gibt einen Überblick über die Präferenzalgebra sowie die Implementierung derselben im Preference Framework.

In Kapitel 3 wird auf die Erweiterungen der Basisklassen des Präferenzframeworks eingegangen. Diese enthielten einige Fehler, die nicht spezifisch für Preference XPath waren. Nichtsdestoweniger sollte man diese vor der eigentlichen Anpassung beheben. Des Weiteren wurde das Präferenzframework um LAYERED-Präferenzen und d -Parameter erweitert.

In Kapitel 4 geht es dann an die Anpassung an JDK 1.5. Nach einem Überblick über die zu lösenden Probleme wird die implementierte Lösung, die das *Adapter*-Pattern benutzt, vorgestellt. Aus verschiedenen Gründen ist es nötig gewesen, mehrere Adapterimplementierungen zu entwickeln. Diese werden am Ende des Kapitels vor- und gegenübergestellt.

Die Evaluation kommt in Kapitel 5 zum Zuge: Hier werden verschiedene Performance-Tests vorgestellt und deren Ergebnisse bewertet.

Kapitel 6 schließlich fasst die Ergebnisse dieser Diplomarbeit kurz zusammen und bietet einen Ausblick auf mögliche zukünftige Änderungen des Preference-XPath-Frameworks.

2 Grundlagen

2.1 Präferenzen

Dieses Kapitel versucht, einen groben Überblick über die Präferenzalgebra zu geben. Für die genauen Definitionen sei auf [Kie02] und [Kie05] verwiesen.

Präferenzen haben häufig die Form „*ich mag A lieber als B*“ und lassen sich daher als Striktordnungen modellieren. In der Regel sind sie allerdings nicht total, d. h. es kann „unvergleichbare“ Elemente geben, die nicht gleich sind, aber von denen trotzdem keines einem anderen vorgezogen wird.

Eine Präferenzabfrage wird immer diejenigen Datensätze liefern, die *optimal* sind, d. h. es gibt keinen Datensatz, der besser ist als einer der Ergebnisdatsätze – diese Semantik wird üblicherweise als *BMO*-Semantik (*Best Matches Only*) abgekürzt (vgl. [Kie02]). Mag beispielsweise jemand rote Autos lieber als blaue, und stehen rote, blaue und schwarze Autos zur Auswahl, so erscheinen im Ergebnis die roten und die schwarzen Autos; die blauen tauchen nicht auf, da es bessere Autos (nämlich rote) gibt. Gäbe es keine roten Autos im Angebot, so würden die blauen Autos im Ergebnis auftauchen.

Zusätzlich ist es für einige Anwendungen noch interessant, ob zwei unvergleichbare Elemente *ersetzbar* sind. Beispielsweise könnte man möglichst billige Autos in rot oder blau suchen. In dem Fall sind **rot** und **blau** besser als alle anderen Farben, aber untereinander unvergleichbar; es steht aber von vorneherein nicht fest, ob man das billigste rote Auto, obwohl es teurer ist als das billigste blaue, nicht trotzdem bevorzugen wird. Sind rot und blau unersetzbar, so werden sowohl das billigste rote als auch das billigste blaue Auto geliefert; sind sie ersetzbar, so taucht nur das billigere der beiden im Ergebnis auf.

Welche Elemente ersetzbar sind, wird durch die so genannte *Ersetzbarkeits-Relation* oder auch *SV-Relation* (für das englische *substitutabile values*, ersetzbare Werte) ausgedrückt. Diese wird in [Kie05] als eine Äquivalenzrelation definiert. Vergleichbare Elemente dürfen hierbei nicht ersetzbar sein (bzw. ersetzbare Elemente müssen unvergleichbar sein), weil sonst Inkonsistenzen auftreten könnten.

Eine Präferenz $P = (A, <_P, \cong_P)$ besteht also aus einer nicht leeren (möglicherweise unendlichen) Menge A , genannt *Domäne*, einer (meist nicht totalen) Striktordnung $<_P$, genannt *Besser-Als-Relation* und einer Äquivalenzrelation $\cong_P$, der *SV-Relation*.

Wir nennen im folgenden die Striktordnung einer Präferenz *negativtransitiv*, falls ihre Negation transitiv ist, d. h. für alle a, b, c gilt: $\neg(a <_P b) \wedge \neg(b <_P c) \Rightarrow \neg(a <_P c)$.

2.1.1 Basispräferenzen

Nachdem die Aufzählung aller Tupel einer Striktordnung (genau wie die Aufzählung aller Äquivalenzklassen einer SV-Relation) recht schnell unübersichtlich oder im Fall einer unendlichen Domäne unmöglich werden kann, kombiniert man in der Praxis einfachere Präferenzen, die so genannten Basispräferenzen, zu komplexeren Präferenzen.

Basispräferenzen untersuchen in der Regel genau eine Eigenschaft der Datensätze, also beispielsweise die Farbe oder den Preis. Theoretisch lässt die Definition es zu, auch Tupel von Eigenschaften in einer Basispräferenz zu untersuchen. Dies führt aber zu unübersichtlicheren Präferenzen als die Kombination mehrerer Basispräferenzen; daher wird davon bei Preference XPath bisher kein Gebrauch gemacht.

Eine Basispräferenz definiert von sich aus keine SV-Relation; SV-Relationen können also fast¹ beliebig mit den Basispräferenzen kombiniert werden.

Wichtige Untertypen der Basispräferenzen sind *kategorielle Präferenzen* und *numerische Präferenzen*.

Kategorielle Präferenzen haben im Normalfall kleine, überschaubare Domänen, wie beispielsweise Farben, Notebookhersteller oder Treibstofftypen bei Autos.

Bei der mächtigsten kategoriellen Präferenz, der so genannten EXPLICIT-Präferenz, werden die Tupel der Besser-Als-Relation explizit angegeben. Zusätzlich zu den angegebenen Tupeln sind alle Elemente, die in keinem Tupel vorkommen, schlechter als alle Elemente, die vorkommen. Offensichtlich kann – wenn die Domäne endlich und bekannt ist – jede andere Basispräferenz auch als EXPLICIT-Präferenz dargestellt werden.

Neben der EXPLICIT-Präferenz, bei der alle Tupel explizit angegeben werden, gibt es hier noch Präferenzen, die allesamt Spezialfälle der (zeitlich später eingeführten) LAYERED-Präferenz sind.

¹Vergleichbare Elemente dürfen, wie oben beschrieben, nicht ersetzbar sein

Die POS-Präferenz (*Positiv*-Präferenz) beispielsweise definiert eine Menge von Elementen, die besser sind als alle anderen Elemente – wobei zwei Elemente innerhalb der Menge sowie zwei Elemente außerhalb der Menge jeweils unvergleichbar sind. Unser Einführungsbeispiel mit den roten oder blauen Autos ist eine $\text{POS}(\text{farbe}, \{\text{rot}, \text{blau}\})$ -Präferenz.

Bei der NEG-Präferenz (*Negativ*-Präferenz) sind die Elemente der angegebenen Menge schlechter als alle anderen.

Die POS/POS-Präferenz erlaubt es, zwei Mengen von Elementen anzugeben – hierbei sind Elemente der ersten Menge besser als die Elemente der zweiten Menge, welche wiederum besser sind als alle anderen Elemente. Elemente innerhalb der Mengen sind auch hier unvergleichbar.

Die POS/NEG-Präferenz schließlich erlaubt es, eine Menge von guten und eine Menge von schlechten Elementen anzugeben. Nicht aufgeführte Elemente liegen dazwischen.

Eine ANTICHAIN-Präferenz ist die einfachste kategorielle Basispräferenz. Hierbei sind alle (verschiedenen) Werte untereinander unvergleichbar. Diese Präferenz macht als alleinige Präferenz keinen Sinn; in einer Kombination von mehreren Basispräferenzen kann sie allerdings nützlich sein.

Eine LAYERED-Präferenz besteht aus einer endlichen Anzahl von disjunkten Mengen, den *Layern*, die eine Partition der Domäne bilden und von denen alle bis auf eine endlich sind (bei einer endlichen Domäne sind alle Mengen endlich). In der Praxis werden alle endlichen Mengen angegeben, statt der unendlichen (oder der größten endlichen) wird das Schlüsselwort **others** benutzt. Elemente der ersten Menge sind besser als Elemente der zweiten Menge, welche wiederum besser sind als Elemente der dritten Menge, und so weiter. Elemente innerhalb einer Menge sind hier natürlich auch unvergleichbar.

Mit einer LAYERED-Präferenz ist es möglich, alle POS-, NEG-, POS/POS-, POS/NEG- und ANTICHAIN-Präferenzen abzubilden. Die Präferenzen $\text{POS/NEG}(\text{farbe}, \{\text{blau}\}, \{\text{rosa}\})$ und $\text{LAYERED}(\text{farbe}, \{\{\text{blau}\}, \text{others}, \{\text{rosa}\}\})$ sind beispielsweise äquivalent – beide bevorzugen Blau vor allen anderen Farben (außer Rosa), und alle diese Farben vor Rosa. Durch Verwendung von mehr als drei Layern kann man auch komplexere Präferenzen darstellen.

Numerische Präferenzen haben als Domäne die reellen Zahlen. Ein Beispiel für eine numerische Präferenz ist die AROUND-Präferenz, die Werte umso mehr bevorzugt, je näher sie an einem bestimmten Wert liegen. $\text{AROUND}(\text{preis}, 20.00)$ wird also Artikel, die exakt 20 Euro kosten, vor Artikeln, die 19,99 Euro oder 20,01 Euro kosten, bevorzugen.

In der Praxis ist es häufig so, dass man aber eine gewisse Toleranz zulassen will. Eine Möglichkeit dafür ist die BETWEEN-Präferenz, die alle Werte zwischen seinen zwei Schranken als gleich gut ansieht. Eine andere Möglichkeit ist der so genannte d -Parameter, mit dem man einen Abstand um das Optimum angeben kann. Werte in diesem Bereich sind zwar schlechter als das Optimum, aber alle gleich gut. Dieser d -Parameter wirkt sich dann auch auf Werte aus, die weiter vom Optimum entfernt liegen, indem alle Werte, die zwischen d und $2d$ entfernt sind, und so weiter, wiederum gleich gut sind. Eine AROUND-Präferenz mit d -Parameter ähnelt also – wie in [Kie05] treffend festgestellt wird – einer zweidimensionalen Zielscheibe.

Verschiebt man eine der beiden Schranken der BETWEEN-Präferenz quasi ins Unendliche, so erhält man eine MORETHAN- bzw. LESSTHAN-Präferenz; alle Elemente größer (bzw. kleiner) als ein Schwellenwert sind gleich gut; die anderen Elemente sind umso besser, je näher sie am Schwellenwert liegen.

Verschiebt man nun diesen Schwellenwert auch noch ins Unendliche, so erhält man eine HIGHEST- bzw. LOWEST-Präferenz. Hierbei sind dann möglichst hohe (bzw. möglichst niedrige) Werte am besten. Das Eingangsbeispiel suchte möglichst billige Autos; dies ist eine LOWEST(*preis*)-Präferenz.

Der d -Parameter kann auch bei diesen numerischen Präferenzen benutzt werden – die HIGHEST- und LOWEST-Präferenzen benötigen hierfür dann aber noch einen Bezugspunkt (der gleichzeitig Supremum/Infimum sein sollte), ab dem die Abstände gemessen werden.

Eine Mischung aus kategorieller und numerischer Präferenz ist die SCORE-Präferenz. Diese kommt mit jeder denkbaren Domäne klar, erfordert aber zusätzlich eine Score-Funktion, die die Elemente der Domäne auf reelle Zahlen abbildet. Elemente mit höherem Score sind besser, Elemente mit gleichem Score unvergleichbar. Durch Definition geeigneter Score-Funktionen lassen sich sämtliche bisher behandelten Präferenzen außer der EXPLICIT-Präferenz als SCORE-Präferenzen darstellen. Dies ist allerdings primär eine theoretische Erkenntnis; in der Praxis wird man natürlich versuchen, die vordefinierten Präferenzen zu nutzen, anstatt sich neue Score-Funktionen zu definieren.

Alle Score-Funktionen (und damit alle in diesem Abschnitt aufgeführten Präferenzen außer der EXPLICIT-Präferenz) sind negativtransitiv.

2.1.2 Basis-SV-Relationen

Ähnlich wie bei den Basispräferenzen ist es bei den SV-Relationen unnötig aufwändig, sämtliche Äquivalenzklassen anzugeben. Es zeigt sich sogar, dass man in der Praxis in

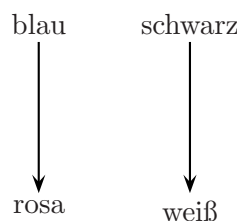
nahezu allen Fällen mit zwei speziellen Formen von SV-Relationen auskommen kann. Dies sind die triviale und die reguläre SV-Relation.

Bei der *trivialen SV-Relation* sind keine (verschiedenen) Elemente ersetzbar; jedes Element besitzt also seine eigene Äquivalenzklasse. Das andere Extrem, die *reguläre SV-Relation*, betrachtet alle unvergleichbaren Elemente als ersetzbar. Sind Zwischenformen gewünscht, so können diese als *explizite SV-Relation* angegeben werden. Generell liefert die reguläre SV-Relation die kleinsten Ergebnismengen.

Allerdings muss man bei der regulären SV-Relation aufpassen: Ist die zugrundeliegende Basispräferenz nämlich nicht negativtransitiv, kann es mit der regulären SV-Relation zu Inkonsistenzen kommen ([Kie05] umgeht das Problem dadurch, dass es die reguläre SV-Relation nur für SCORE-Präferenzen definiert).

Betrachten wir hierzu ein einfaches Beispiel:

EXPLICIT('rosa' < 'blau', 'weiß' < 'schwarz')



Hier sind offensichtlich blau und schwarz unvergleichbar, rosa und schwarz auch. Bei der regulären SV-Relation müssten diese drei Farben alle in eine Äquivalenzklasse. Nachdem aber blau und rosa vergleichbar sind, dürfen sie nicht in derselben Äquivalenzklasse landen, was einen Widerspruch darstellt. Die reguläre SV-Relation ist für diese Präferenz also nicht definiert.

Abbildung 2.1:
Eine EXPLICIT-Präferenz

2.1.3 Zusammengesetzte Präferenzen

Zur Kombination von Basispräferenzen stehen zusammengesetzte Präferenzen zur Verfügung. Grundsätzlich leiten zusammengesetzte Präferenzen ihre SV-Relation immer aus ihren Teilpräferenzen ab; es sind also keine speziellen SV-Relationen für die zusammengesetzten Präferenzen nötig.

In [Kie02] werden (außer der RANK-Präferenz, die nur SCORE-Präferenzen kombinieren kann) insgesamt fünf verschiedene zusammengesetzte Präferenzen beschrieben. Davon unterstützt das Preference-Framework jedoch nur zwei (Pareto-Präferenz und Priorisierung), die ich im folgenden beschreiben werde.

Name	Farbe	Preis
A	schwarz	20000
B	rot	35000
C	schwarz	32000
D	rot	15000
E	gelb	40000
F	blau	42000

Tabelle 2.1: Ausschnitt einer Beispieldatenbank

Die anderen drei Präferenzen werden im Rahmen der Definition sowie für einige Präferenz-Umformungen und Beweise benötigt, kommen in der Praxis aber kaum vor. Zudem kann man diese in den meisten Fällen auch durch (evtl. andere) Basispräferenzen in Verbindung mit den beiden unterstützten zusammengesetzten Präferenzen ausdrücken.

Im Folgenden werde ich als Beispiel immer die aus dem Einführungsbeispiel bekannten Präferenzen $\text{POS}(\text{farbe}, \{\text{rot}, \text{blau}\})$ und $\text{LOWEST}(\text{preis})$ benutzen, erstere mit verschiedenen SV-Relationen.

Die *Pareto-Präferenz* $\otimes$ verbindet mehrere Präferenzen, wobei jede davon gleich wichtig ist. Sie stellt also das weiche Gegenstück zu den Und- und Oder-Abfragen in klassischen Datenbanken dar. Zwei Elemente sind gleich, wenn sie bezüglich beider Präferenzen gleich sind. Ein Element ist besser als ein anderes, wenn es bezüglich einer der Basispräferenzen echt besser ist, und bezüglich der anderen gleich, besser oder ersetzbar. Es ist ersetzbar, falls es bezüglich aller Basispräferenzen ersetzbar ist. Wenn wir in unserem Beispiel eine Pareto-Präferenz verwenden, und in der Datenbank (vgl. Tabelle 2.1 ein billiges schwarzes (A) und ein teures rotes Auto (B) ist, so werden beide als Ergebnis geliefert. Ein zusätzlich vorhandenes teures schwarzes (C) hingegen wird nicht erscheinen. Gäbe es ein billiges rotes Auto D (das billiger ist als das billige schwarze), so würde es als einziges Ergebnis auftauchen. Ob ein noch teureres gelbes Auto (E) im Ergebnis erscheinen würde, hängt von der SV-Relation der POS-Präferenz ab. Ist diese trivial, so wird es erscheinen, ist diese regulär (und somit schwarz und gelb ersetzbar), nicht.

Bei der *Priorisierung* $\&$ hingegen ist eine der Präferenzen wichtiger als die andere. Ein Element ist hier besser als ein anderes, wenn es bezüglich der ersten Präferenz besser ist oder wenn es bezüglich der ersten Präferenz ersetzbar und bezüglich der zweiten Präferenz besser ist. Gleichheit und Ersetzbarkeit sind hier genauso definiert wie bei der Pareto-Präferenz. In unserem Beispiel – wenn wir $\text{POS}(\text{farbe}, \{\text{rot}, \text{blau}\}) \& \text{LOWEST}(\text{preis})$ auswerten – erhalten wir (vom teuren ro-

ten und billigen schwarzen Auto) nur das teure rote Auto. Gibt es zusätzlich noch ein noch teureres blaues Auto (F), so erscheint es bei trivialer SV-Relation, bei regulärer aber nicht.

Verwendet man bei einer Pareto-Präferenz oder Priorisierung als erste Präferenz eine ANTICHAIN-Präferenz mit trivialer SV-Relation, so entspricht dies in etwa der aus SQL bekannten Gruppierung. Die Anfrage $\text{ANTICHAIN}(\text{farbe}) \otimes \text{LOWEST}(\text{preis})$ wird für jede Farbe das billigste verfügbare Auto ausgeben.

Natürlich kann man Pareto-Präferenzen und Priorisierungen auch auf weitere zusammengesetzte Präferenzen anwenden und nicht nur auf Basispräferenzen. Beide kombinierten Präferenzen sind dabei assoziativ, die Pareto-Präferenz zusätzlich kommutativ. Jede ANTICHAIN-Präferenz mit regulärer SV-Relation ist neutrales Element beider kombinierter Präferenzen.

2.2 XPath

Die XML Path Language (XPath, vgl. [W3C99]) stellt eine bequeme Möglichkeit dar, XML-Fragmente aus einer XML-Datei abzufragen, die bestimmten (*harten*) Kriterien genügen. XPath-Ausdrücke sind einfache Strings (und nicht etwa wieder XML); daher können sie in vielen Anwendungsgebieten problemlos eingesetzt werden. XPath erlaubt allerdings nur reine Abfragen, keine Veränderung der Daten.

XPath-Ausdrücke werden immer im Bezug auf einen Kontextknoten (ein gewähltes Element des XML-Dokuments) ausgewertet. Sie beginnen meist mit einem *Pfad*, der angibt, wie man vom Kontextknoten die potenziellen Ergebnisse erreichen kann. Die einfachste Form des Pfades erinnert an Pfade in Dateinamen oder URLs; „./db/klasse“ wählt zum Beispiel alle <klasse>-Elemente aus, die Kind eines <db>-Elementes sind; letzteres Element ist wiederum Kind des Kontextknotens. Komplexere Formen von Pfaden verwenden ../ für den übergeordneten Knoten, @attr um das Attribut attr eines Knotens zu selektieren oder *Achsbeschreibungen*, die genauer festlegen, über welche der 13 verfügbaren *Achsen* (wie beispielsweise *Kinder*, *Nachkommen*, *Eltern*, *Vorfahren*, *frühere Geschwister*, *spätere Geschwister*, *Vorher* (in Dokumentreihenfolge), *Attribute* etc.) man den Knoten vom aktuellen Kontextknoten auf welche Weise erreichen kann. Das zweite wesentliche Element von XPath sind (harte) Bedingungen. Diese schränken die über den Pfad selektierten Knoten über boolesche Bedingungen ein. Bedingungen werden hinter den Pfad in eckige Klammern geschrieben und dürfen ihrerseits wieder Pfade oder andere Bedingungen enthalten.

Da das Ergebnis von Bedingungen wieder Teil eines Pfades sein kann, kann man somit beliebig komplexe Ausdrücke erzeugen:

```
/db/schule[@typ='Gymnasium']/klasse[@klassenstufe < 8 AND  
  schueler[@klassensprecher='yes']/@nachname='Huber']/schueler[1]
```

selektiert in einer fiktiven Schuldatenbank jeweils den ersten Schüler (in Dokumentreihenfolge) jeder Unterstufenklasse, die in einer Schule mit Typ „Gymnasium“ liegt und dessen Klassensprecher (Schüler, dessen Attribut `klassensprecher` den Wert `yes` aufweist) mit Nachnamen Huber heißt.

XPath-Parser sind in dem Sinne *erweiterbar*, dass eine Anwendung eigene Funktionen definieren kann, die über einen Namespace adressiert und entweder auf oberster Ebene oder innerhalb einer Bedingung (nicht aber innerhalb eines Pfades) aufgerufen werden können. Dies erlaubt es, XPath einfacher in bestehende XML-basierte Systeme oder Sprachen einzubetten.

Reines XPath kann allerdings keine weichen Bedingungen (Präferenzen) modellieren; deshalb wurde Preference XPath entwickelt.

Dort kann anstelle jeder harten Bedingung (die in eckige Klammern eingeschlossen wird) auch eine weiche Bedingung stehen (die in `#[` und `]#` eingeschlossen wird). Weiche Bedingungen besitzen eine eigene, nicht XPath-typische Syntax. Basispräferenzen haben meist folgendes Aussehen: Zunächst gibt ein XPath-Unterausdruck an, auf welchen Teil des Datensatzes die Präferenz angewandt werden soll. Das darauffolgende Schlüsselwort gibt den Typ der Präferenz an. Eventuelle zusätzliche Parameter und SV-Relationen folgen dann auf dieses Schlüsselwort. Pareto-Präferenzen werden durch das Schlüsselwort „*and*“ verbunden, Priorisierungen durch „*prior to*“.

Unser Auto-Beispiel (möglichst billige rote oder blaue Autos) würde man in Preference XPath also als

```
/autos/modell#[farbe in ('Rot', 'Blau') regular prior to preis lowest]#
```

formulieren.

Eine genauere Syntaxbeschreibung findet sich im XPath-Tutorial ([LDI06]).

2.3 Das Preference-Framework

Dieses Kapitel versucht, einen kurzen Überblick über den Aufbau des Preference-Framework zu geben. Teile, an denen im Rahmen dieser Diplomarbeit Änderungen vorgenommen wurden, werden in dem Zustand beschrieben, den sie zu Beginn der Diplomarbeit hatten.

Da das Preference-Framework verschiedene Abfragesprachen unterstützt, besteht es aus den *Basisklassen*, die von der Abfragesprache unabhängig sind, und aus den Klassen, die jeweils nur für XPath oder SQL zuständig sind.

Nachdem die Basisklassen und die sprachspezifischen Klassen in der Version, die ich vom Lehrstuhl erhalten habe, zusammen in einem Verzeichnis lagen, referenzierten sie sich stark gegenseitig (wenn auch nur über unbenutzte Imports oder nie benutzte Attribute). Diese überflüssigen Referenzen wurden entfernt, so dass die Basisklassen ohne den Rest compilierbar sind; ebenso ist Preference XPath nun auch ohne die grafische Test-Oberfläche compilierbar und einsetzbar.

Des Weiteren enthielt die Codebasis etliche Klassen und Datendateien, die nur für Preference SQL relevant sind – diese wurden aus meiner Codebasis entfernt. Nachdem ein anderer Diplomand zeitgleich mit mir eine Diplomarbeit über Preference SQL schreibt, haben wir die Arbeit an den Basisklassen über ein gemeinsam genutztes Versionsverwaltungssystem (CVS) untereinander abgestimmt.

2.3.1 Die Basisklassen

Die verschiedenen Präferenzen implementieren alle das *IPreference*-Interface. Dieses schreibt unter anderem eine Methode vor, die Objekte bezüglich dieser Präferenz vergleichen kann. Nachdem die Basisklassen mit jeder Abfragesprache funktionieren sollen, besitzt jede Präferenz eine Referenz auf einen *Attributselektor*, dessen Aufgabe darin besteht, aus einem Datensatz (und einem *Abfragekontext*) für diese Abfragesprache ein Attribut zu ermitteln. Jede Abfragesprache implementiert also ihren eigenen Attributselektor, der dann in allen Präferenzen verwendet wird. (Diese Konstruktion entspricht im Wesentlichen dem *Bridge*-Pattern aus [GHJV94].) Ein Präferenzobjekt repräsentiert damit sowohl die Art der Präferenz (also beispielsweise „rot bevorzugt“) als auch (über den Attributselektor), auf welches Datenfeld diese angewandt werden soll.

Das Vergleichsergebnis wird durch einen der Werte aus Tabelle 2.2 angegeben.

Konstante	Bedeutung
LESS	A ist schlechter als B, also $A <_P B$
GREATER	A ist besser als B, also $B <_P A$
EQUAL	$A = B$
UNRANKED	A und B sind unvergleichbar
SUBSTITUTABLE	A und B sind ersetzbar

Tabelle 2.2: Vergleichsergebnisse der Präferenzen

Prinzipiell können Präferenzobjekte auf zwei verschiedene Weisen entstehen:

- Durch Parsen eines Ausdrucks in einer Abfragesprache, wie es bei Preference XPath und Preference SQL der Fall ist.
- Algorithmisch, beispielsweise beim *Preference Miner* durch Analysieren bestehender Kundendaten.

Gerade in letzterem Fall ist es wichtig, dass man diese (letztendlich als Objekt vorliegenden) Präferenzen persistent abspeichern kann. Zudem kann es auch für Debuggingzwecke nützlich sein, in eine Präferenz hineinsehen zu können. Für diese Zwecke gibt es das Preference-Repository, das XML-Fragmente (im einfachsten Fall in einer Datei gespeichert) persistent festhält. Um dies zu ermöglichen, ist es wichtig, dass jedes Präferenzobjekt in ein derartiges XML-Fragment serialisiert und wieder deserialisiert werden kann. Diese Funktionalität stellen zwei Klassen (**AlgebraTreeWalker** und **XMLPrefTreeWalker**) zur Verfügung; in der Ursprungsversion mussten beide Klassen alle möglichen Präferenzen „kennen“, um sie serialisieren und deserialisieren zu können. Das Erzeugen neuer Präferenzen (die nicht über den Extension-Mechanismus angesprochen werden) erforderte also, dass diese beiden Klassen auch mitangepasst werden – was man leicht vergessen konnte.

Stichwort Extension-Mechanismus: Um es für Spezialanwendungen einfach möglich zu machen, eigene Präferenzen zu verwenden, ohne den Parser anzupassen, verfügt das Preference-Framework über die Möglichkeit, eigene Präferenzklassen dynamisch zur Laufzeit über Reflection nachzuladen und zu verwenden. Dafür muss in der Anfrage jeweils der komplette Klassenname der Extension angegeben werden.

Des Weiteren besteht die Möglichkeit, so genannte Score- und Rank-Funktionen zu definieren und diese zur Laufzeit zu laden.

Eine Score-Funktion ist eine Java-Klasse, die jedem Wert der Domäne einen numerischen Score zuordnet; Elemente mit höheren Scores werden bevorzugt.

Eine Rank-Funktion ist eine besondere Score-Funktion, da sie ihren Score auf der Basis der Ergebnisse anderen Score-Funktionen ermittelt – also beispielsweise `(score1 + score2) * score3`. Diese Unter-Score-Funktionen werden dabei in der Preference-Query angegeben, sind also unabhängig von der Rank-Funktion austauschbar.

Diese Score- und Rank-Funktionen werden dann von der SCORE-Präferenz verwendet.

Nachdem diese beiden Interfaces in meiner Diplomarbeit nicht verändert wurden, sei für die genaue Beschreibung auf das Preference-XPath-Tutorial (siehe [LDI06]) verwiesen.

Sowohl SV-Relationen (außer der trivialen) als auch *d*-Parameter wurden vom Preference-Framework nur über Extensions unterstützt – allgemeine LAYERED-Präferenzen gar nicht.

Jede Präferenz verfügt des Weiteren über eine `clonePreference`-Methode, die eine tiefe Kopie dieser Präferenz anlegt. Dies ist insbesondere dann wichtig, wenn man den Attributselektor einer bereits erzeugten Präferenz austauschen will, ohne die Originalpräferenz zu verändern. Von dieser Möglichkeit wurde im XPath-Teil reichlich Gebrauch gemacht.

2.3.2 Preference XPath

Wie in 2.2 bereits erwähnt, ermöglicht XPath in einfacher und standardisierter Weise, komplexe harte Abfragen auf XML-Dokumenten durchzuführen; Preference XPath erweitert diese Sprache um *weiche Constraints*, die es ermöglichen, auf bequeme Weise präferenzbasierte Abfragen auszuführen.

Zusätzlich zu der Möglichkeit, Preference-XPath-Anfragen auf (im RAM befindliche) XML-Dokumente anzuwenden, besteht auch eine Anbindung an die XML-Datenbank Apache Xindice. Allerdings funktioniert Xindice (in der momentan vorliegenden Version) ohnehin nur unter Java 1.4.x und ist daher für meine Diplomarbeit weniger relevant. Abgesehen von Anpassungen, die nötig sind, um die Xindice-Anbindung lauffähig zu halten, wird hier nicht weiter darauf eingegangen.

Preference XPath baut auf den Basisklassen auf. Eingegebene XPath-Ausdrücke werden mit einem JavaCC-Parser geparkt und in reines XPath umgeschrieben. Rein in dem Sinne, dass alle weichen Bedingungen in Aufrufe der `pref:PREFERENCE`- und `pref:LOCPATH`-Funktionen umgewandelt werden.

Normalerweise (bei Anfragen auf einem einzigen XML-Dokument) wird nur die erste der beiden verwendet, die zwei Parameter erhält:

`pref:PREFERENCE(path, index)`

`path` gibt den XPath-Pfad zu dem Datenfeld an, auf das die Präferenz angewandt werden soll. Die Präferenz selbst wird in einem Präferenzarray gespeichert; `index` verweist auf das Präferenzobjekt in diesem Array. Jeder geparste Ausdruck bekommt sein eigenes Präferenzarray, das zusammen mit dem umgeschriebenen XPath-Ausdruck das „Rewriting“ darstellt.

Eine Anfrage wie

```
/fallschirm#[(@sicherheit highest and @platzbedarf lowest)
              prior to @preis lowest]#/@bestellnummer
```

erzeugt dabei ein Rewriting, das sich folgendermaßen in XML serialisieren lässt:

```
<?xml version="1.0" encoding="UTF-8"?>
<QUERYREWRITING QUERY="pref:PREFERENCE(/fallschirm,0)/@bestellnummer">
  <PREFERENCE INDEX="0">
    <PRIORITIZATION>
      <PARETO>
        <EXTREMAL SELECTOR="@sicherheit" TYPE="HIGHEST"/>
        <EXTREMAL SELECTOR="@platzbedarf" TYPE="LOWEST"/>
      </PARETO>
      <EXTREMAL SELECTOR="@preis" TYPE="LOWEST"/>
    </PRIORITIZATION>
  </PREFERENCE>
</QUERYREWRITING>
```

Laufen Anfragen über mehrere XML-Dokumente gleichzeitig (wie es bei der Anbindung an Xindice der Fall ist), wird zusätzlich noch die `LOCPATH`-Funktion benutzt. In dem Fall werden die längsten Teilausdrücke, die direkt auf der Datenbank ausgewertet werden können (also alles „vor“ der jeweils ersten Präferenz) im `Locpath`-Array festgehalten (würden sie direkt als Parameter übergeben, so würden sie von XPath schon vor Aufruf der Funktion ausgewertet). Die `LOCPATH`-Funktion erhält nun den Index für das `Locpath`-Array und einen Index für das Präferenzarray als Parameter übergeben. Denn auf das Ergebnis eines `Locpath`-Eintrags muss als nächstes immer eine Präferenz angewandt werden – andernfalls könnte man den `Locpath`-Eintrag länger machen. In der Praxis tritt immer mindestens ein `LOCPATH`-Eintrag auf – reine XPath-Anfragen werden von Xindice ohne Verwendung von `Preference` XPath geparst. Mehr als ein

LOCPATH-Eintrag kann bei der Verwendung des Vereinigungsoperators auftreten, ist in der Praxis aber eher selten anzutreffen.

Die umgeschriebene XPath-Anfrage wurde bisher an den Xalan-XPath-Parser, der bei Java 1.4.1 mitgeliefert wird (aber nicht Teil der dokumentierten API ist), übergeben. Hierbei wurde ein modifizierter Prefix-Resolver verwendet, der den Präfix `pref` auf einen besonderen URI auflöst. Dieser URI wird von Xalan so interpretiert, dass in der Klasse `preference.xpath.PreferenceFunc` nach statischen Methoden namens `PREFERENCE` und `LOCPATH` gesucht wird, die zum Auswerten aufgerufen werden.

Ein Problem dieses Ansatzes (wir werden später noch weitere kennenlernen) besteht darin, dass in statischen Methoden nur die Parameter der XPath-Funktion (also Node-Sets bzw. Zahlen) übergeben werden, insbesondere also nicht das Preference-Array und das Locpath-Array. Um dieses Problem zu umgehen haben die Autoren des Preference Frameworks zu einem „Trick“ gegriffen: Java bietet seit 1.2 so genannte *ThreadLocals* – Variablen, die je nach aufrufendem Thread verschiedene Werte annehmen. Solche ThreadLocals werden in der Klasse `preference.xpath.PXPathImpl` in statischen Klassenattributen vorgehalten, um das Preference-Array und das Locpath-Array aufzunehmen. Bei jedem Aufruf einer Methode auf dem XPath-Objekt (genaugenommen einer Unterklasse davon) werden die ThreadLocals auf die jeweiligen Werte für diese Query gesetzt. (Diese Implementierung war auch noch dahingehend fehlerhaft, dass in bestimmten Situationen die Thread-Locals erst zu spät gesetzt wurden, wodurch manche Anfragen erst beim zweiten Versuch das gewünschte Ergebnis brachten.)

Die `PREFERENCE`- und `LOCPATH`-Funktionen bauen die Ergebnismenge inkrementell auf. Jedes Element wird mit der momentanen Ergebnismenge verglichen; schlechtere Elemente werden gar nicht erst aufgenommen, bessere Elemente entfernen die schlechteren Elemente. Hierzu müssen auf den Elementen wiederum XPath-Anfragen (*Unteranfragen*) ausgewertet werden, die die zu vergleichenden Attribute aus dem XML-Element extrahieren.

Nachdem ein Thread gleichzeitig nur einen Ausdruck auswerten kann, könnte man annehmen, dass diese Vorgehensweise in allen Fällen den gewünschten Effekt hat. In der Praxis ist dies ab Java 1.4.2 aber leider nicht mehr der Fall, da dort beim Aufruf der `evaluate`-Methoden zunächst nur so viel berechnet wird, wie für die Bestimmung der Größe des Ergebnisses nötig ist. Die einzelnen Werte werden erst dann ausgewertet, wenn auf dem `ResultSet` die passende `item`-Methode aufgerufen wird. Wertet man nun zwei verschiedene Queries zunächst aus und durchläuft diese danach synchron, so werden die Werte der ersten Anfrage ggf. im Kontext der zweiten ermittelt.

Abbildung 2.2 zeigt in einem Sequenzdiagramm, wie eine Preference-XPath-Anfrage in etwa ausgewertet wird. Bei der Erstellung dieses Diagramms wurden unwichtige

Details, die nicht an einer *Systemgrenze* stattfinden, weggelassen, um es nicht allzu unübersichtlich zu machen.

Als ersten Schritt schreibt der Benutzer (also das Programm, das Preference-XPath benutzt) durch Aufruf von *compile()* die Anfrage in eine reine XPath-Anfrage um. Das dazugehörige Preference-Array enthält Präferenzen, die den Preference-XPath-Attributselektor für das Auswerten der Attribute verwenden.

Als nächstes wertet der Benutzer diese umgeschriebene XPath-Anfrage auf einem Knoten eines DOM-Dokumentes aus. Diese Anfrage wird an einen für Preference XPath konfigurierten (**PREFERENCE** und **LOCPATH** sind registriert) XPath-Parser weitergegeben. Dieser beginnt nun, die Anfrage zu parsen. Bei diesem Parsen wird er früher oder später auf einen Aufruf der **PREFERENCE**-Funktion stoßen, die er selbst nicht auswerten kann. Deshalb ruft der Parser hierzu eine Methode von Preference XPath auf (Ort und Signatur dieser Methode hängen vom jeweiligen XPath-Parser ab), die eine Liste der potenziellen Ergebnisknoten sowie einen Index ins Präferenz-Array erhält.

Diese Methode legt nun ein neues, leeres BMO-Set für die gewünschte Präferenz an und fügt der Reihe nach alle potenziellen Knoten dort hinzu. Das BMO-Set selbst wird beim Einfügen eines neuen Knotens gegebenenfalls schlechtere Knoten entfernen oder einen einzufügenden schlechten Knoten komplett abweisen. Hierzu muss das BMO-Set die Attributselektoren der einzelnen Elemente auswerten; nachdem sich die Werte nicht ändern, muss jeder Attributselektor für jedes Element nur einmal ausgewertet werden (obwohl auf dessen Wert im schlechtesten Fall so oft zugegriffen wird, wie es potenzielle Ergebnisse gibt).

Das eigentliche Auswerten des Attributselektors erfolgt durch Erzeugen einer XPath-Unterabfrage auf dem jeweiligen Knoten. Diese Unterabfragen einer Präferenz verwenden alle denselben Attributselektor (also denselben XPath-String) und das selbe DOM-Dokument als Grundlage; der Kontextknoten hingegen variiert. Auch die Unterabfragen können theoretisch wieder Präferenzausdrücke (deren Präferenzen vom Parser andere Indizes erhalten haben) enthalten; dies führt dann dazu, dass auch diese wiederum die **PREFERENCE**-Methode aufrufen werden.

Sind die Attributselektoren alle ausgewertet und alle schlechten Ergebnisknoten entfernt, so gibt die Preference-Methode die übrigen (d. h. besten) Ergebnisse an den Parser zurück, der diese – je nach Anfrage – gegebenenfalls weiterparst (Teile selektiert etc.). Das Ergebnis des Parsers (der bei mehreren Präferenzen die Preference-Methode mehrfach aufruft) wird schließlich an den Benutzer zurückgegeben.

Xalan verwendet für die Repräsentation von XML-Fragmenten das so genannte *Document Type Model*, kurz *DTM* ([ASF05b]). Dieses erzeugt im Gegensatz zu *DOM* (vgl. [W3C98]) nicht für jeden Knoten ein eigenes Objekt, sondern stellt diese durch

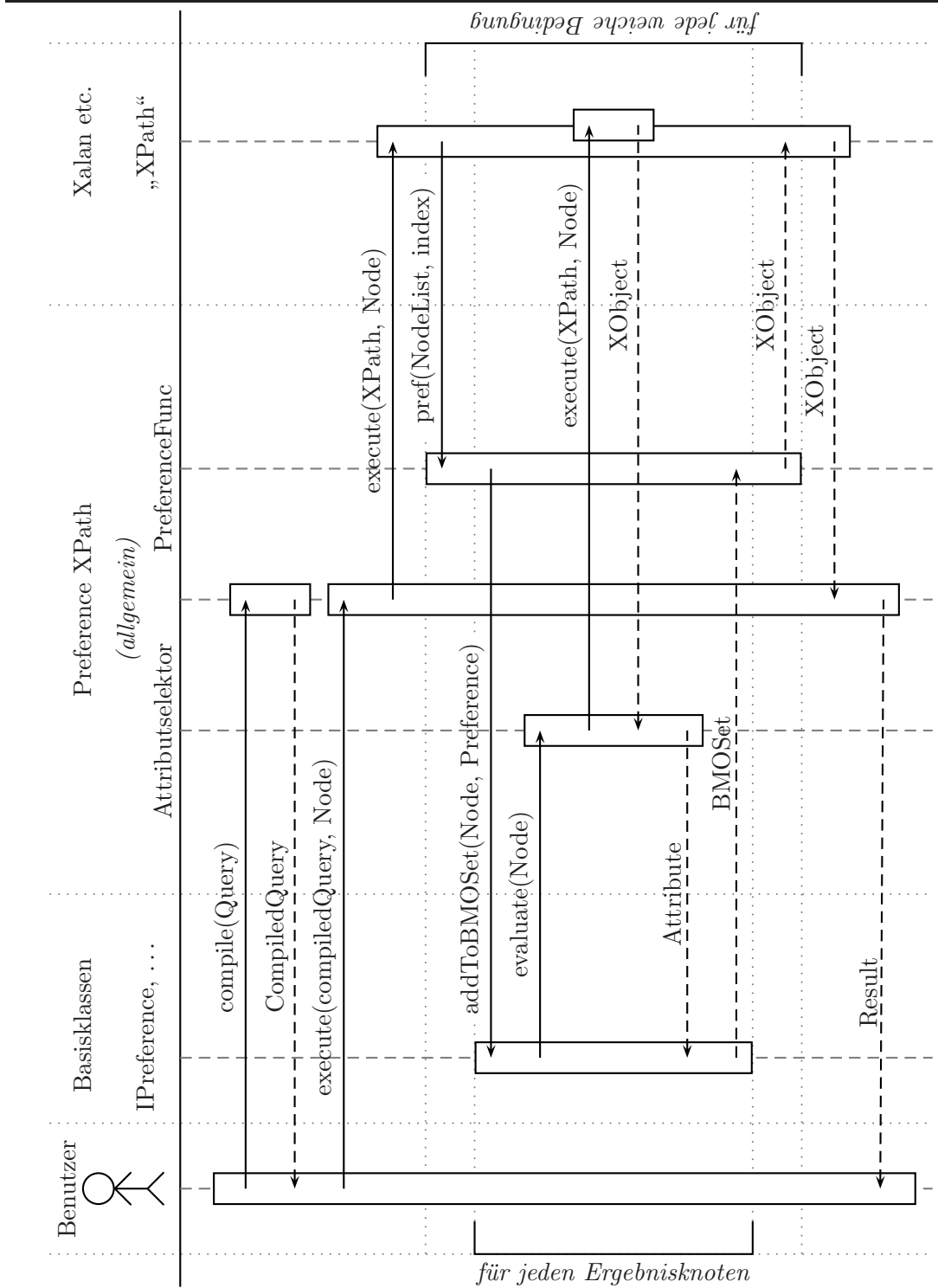


Abbildung 2.2: Schematische Auswertung einer Preference-XPath-Anfrage

eine ganze Zahl (*Handle* genannt) im Bezug auf einen *Kontext* dar, der von mehreren DTM-Knoten verwendet wird. Anstatt Operationen direkt auf den DOM-Knoten auszuführen, werden sie auf dem DTM-Kontext ausgeführt, wobei jeweils noch das *Handle* als zusätzlicher Parameter mitgegeben wird. Dies führt durch die geringere Objektanzahl dazu, dass reine XPath-Anfragen deutlich schneller ausgeführt werden können (und weniger Garbage Collection benötigen) als es bei DOM der Fall wäre. Um die Performance weiter zu erhöhen, wurde bei der Erstellung von Unteranfragen derselbe DTM-Kontext verwendet wie bei der Hauptanfrage (schließlich befindet sich der Bezugsknoten für die neue Anfrage bereits im DTM-Kontext). Dies hatte aber wiederum zur Folge, dass fast der gesamte XPath-spezifische Teil der API von Xerces-spezifischen Klassen abhing.

Zudem tauchten Xerces-Klassen wie `XPathContext`, `PrefixResolver` und natürlich `XObject` (das das Ergebnis einer XPath-Anfrage repräsentiert) in der öffentlichen API von Preference XPath auf. Nachdem diese Klassen in der dokumentierten API logischerweise nicht existieren, und auch in der undokumentierten API ihr Paket gewechselt haben (unter Java 1.4.x befanden sie sich in `org.apache.xpath.*`, seit 1.5 allerdings unter `com.sun.org.apache.xpath.internal.*`), wird sich im Rahmen dieser Diplomarbeit auch die öffentliche API von Preference XPath grundlegend ändern.

Eine weitere Besonderheit von Preference XPath besteht darin, dass in XML (im Gegensatz zum relationalen Datenbankmodell) Attribute mehrfach auftreten können. Die Präferenzen müssen also auf eine Mengensemantik erweitert werden, vgl. [Haf01]. Nachdem dies unabhängig von den einzelnen Präferenzen möglich ist, da die Präferenzbeziehung einer Menge aus der Präferenzbeziehung ihrer Elemente herleitbar ist, wird diese Erweiterung in einer Decorator-Klasse `MultiSetPreference` implementiert; sämtliche Basispräferenzen werden also vom XPath-Parser in `MultiSetPreference`-Objekte verpackt. Diese Klasse wertet in ihrer `evaluate`-Methode zunächst den Attributselektor ihrer Basispräferenz aus. Der Rückgabewert dieses Selektors (der eine Menge darstellt) wird nun elementweise von der Basispräferenz verglichen. Hierbei darf dann die Basispräferenz aber nicht erneut den Attributselektor auf die (bereits selektierten) Mengenelemente anwenden. In der vorliegenden Version wird das dadurch gelöst, dass der Attributselektor temporär mit einem *Dummy*-Attributselektor überschrieben wird, der nichts weiter tut, als die Elemente ohne Veränderung zurückzugeben.

Diese Mengensemantik besteht darin, dass mehrere identische Tags als *Alternativen* aufgefasst werden. Ein Auto mit mehreren `<farbe>`-Tags ist also in mehreren Farben erhältlich, ein Auto mit mehreren `<motor>`-Tags mit verschiedenen Motoren. Jedes einzelne Exemplar hat dabei allerdings immer noch lediglich eine Farbe und einen Motor. Aus unserer Intuition folgt also, dass ein Auto, das als einziges in der Lieblingsfarbe erhältlich ist, besser sein soll als alle anderen Autos, selbst dann, wenn es zusätzlich

noch in einer unbeliebten Farbe erhältlich ist. Genau so verhält sich auch Preference XPath.

Wichtig wird dieses Verhalten beim Datenbankdesign. Würden in einer Datenbank identische Tags nicht als Alternativen, sondern als *Bestandteile* aufgefasst, so ist diese in Preference XPath nicht verwendbar. Eine Datenbank einer Gaststätte könnte beispielsweise all ihre Menüs in `<menue>`-Tags speichern, wobei die einzelnen Gänge des Menüs als `<gang>`-Tags gespeichert werden. In diesem Fall würde Preference XPath die Kundenwünsche nicht adäquat darstellen können, da ein Menü, dessen Hauptgericht der Leibspeise entspricht, dessen Vorspeise aber ungerne verzehrt wird, als bestes Ergebnis aufgefasst würde. Eine solche Datenbank sollte also für die einzelnen Gänge verschiedene Tags (`<vorspeise>`, `<hauptgericht>` usw.) benutzen.

Diese Semantik kann aber auch unerwartete Effekte verursachen, da Tags in XML geschachtelt sein können. Ein Motor kann also wiederum mehrere Eigenschaften haben, die gegebenenfalls doppelt auftauchen können. Kombiniert man nun mehrere Motoreigenschaften über zusammengesetzte Präferenzen, so entspricht das Ergebnis möglicherweise nicht den Erwartungen. Es könnte beispielsweise sein, dass ein billiges Auto mit einem extrem leistungsstarken Benzinmotor oder einem extrem leistungsschwachen Dieselmotor angeboten wird. Nun kommt ein Kunde, der ein möglichst günstiges Modell mit einem leistungsstarken Dieselmotor kaufen will. Stellt dieser nun die Anfrage

```
/AUTOS/MODELL#[PREIS lowest and MOTOR/TREIBSTOFF is 'DIESEL'  
and MOTOR/LEISTUNG highest]#
```

so erhält er dieses Auto in der Ergebnismenge, obwohl es nicht mit einem leistungsstarken Dieselmotor, sondern nur alternativ mit einem Dieselmotor oder einem leistungsstarken Motor erhältlich ist. Er wollte aber ein Automodell, dessen Motor beide Eigenschaften erfüllt.

Um dies ausdrücken zu können, gibt es die NESTED-Präferenz. Hierbei kann man einen Teil der Abfrage auf Unterelementen des eigentlichen Datensatzes ausführen; dadurch wird das obige Problem umgangen. NESTED-Präferenzen werden in Preference XPath durch geschweifte Klammern ausgedrückt. Die Anfrage

```
/AUTOS/MODELL#[PREIS lowest and MOTOR{TREIBSTOFF is 'DIESEL'  
and LEISTUNG highest}]#
```

liefert also genau das, was der Kunde eigentlich hätte haben wollen.

3 Erweiterung der Basisklassen

Zusätzlich zur Umstellung auf Java 1.5 wurden auch einige kleinere Änderungen an den Basisklassen vorgenommen.

3.1 Fehlerbehebung

Während der Umstellung bzw. des Debuggings traten einige Fehler der Basisklassen zutage, die sogleich behoben wurden:

Zusammengesetzte Präferenzen speicherten ihre Teilpräferenzen in Arrays mit fester Größe 10 ab. Diese nirgendwo dokumentierte Beschränkung führte bei längeren Pareto-Präferenzen zu Exceptions. Um das Problem zu beheben, wurde das Array durch eine dynamisch wachsende Liste ersetzt.

Des Weiteren war die bereits beschriebene `clonePreference`-Methode bei einigen Präferenzen fehlerhaft implementiert – entweder gab die Methode `null` zurück, oder sie vergaß, einige Parameter zu initialisieren oder den zugrundeliegenden Attributselektor mitzuklonen. Diese Fehler wurden behoben – jetzt gibt die Methode in jedem Fall einen richtig initialisierten Klon der Präferenz zurück.

Für Testzwecke stellt die Klasse `DefaultDOMDocument` eine Implementierung eines Preference-Repositories zur Verfügung, die die Präferenzen lediglich im Speicher vorhält und nicht auf Festplatte ablegt. Allerdings hat man bei der Erzeugung des DOM-Dokuments vergessen, ein Wurzelement zu erzeugen, wodurch jedes Einfügen einer Präferenz zu einem Fehler führte und die Klasse praktisch unbenutzbar wurde. Nachdem diese Klasse anscheinend bisher nie verwendet wurde, fiel der Fehler erst auf, als ich sie zum ersten Mal benutzte.

3.2 Vorbereitungen

Die Basisklassen-Pakete enthielten einige Interfaces wie `IDomainX`, die weder von Preference SQL noch von Preference XPath gebraucht wurden. Diese wurden aus der Codebasis entfernt.

Des Weiteren waren auch einige Änderungen in den `BMOSet`-Klassen (`BMO = Best Matches Only`) vorzunehmen. Diese Mengen haben die Fähigkeit, beim Einfügen eines Elements immer nur die besten Elemente (bezüglich einer Präferenz) zu behalten und die anderen automatisch zu entfernen. Zusätzlich bieten sie die Möglichkeit, ein bestehendes BMO-Set bezüglich einer anderen Präferenz zu „reduzieren“, d. h. alle Elemente zu entfernen, die bezüglich dieser Präferenz nicht optimal sind.

Allerdings wurde nicht in allen Fällen, bei denen nur die besten Elemente behalten werden sollen, auch `BMOSet`s benutzt. Dies lag daran, dass die Mengen, in denen die besten Element behalten werden sollten, keine expliziten `Collections` waren, sondern beispielsweise als Liste von DTM-Indizes gespeichert wurden. In all diesen Fällen wurde der Kerncode von `BMOSet` per Copy&Paste in neue Klassen übernommen und geeignet angepasst. Um die Codeduplizierung zu reduzieren wurde dieser Code zunächst in eine abstrakte Template-Klasse `AbstractBMOSet` ausgelagert. Diese enthält eine *Templatemethode* (vgl. [GHJV94]) `maybeAddNode`, die sich auf abstrakte Operationen zum Iterieren über die Menge, zum Hinzufügen von Elementen sowie zum Durchführen eines Vergleiches abstützt. Dadurch konnte diese Funktionalität, die ansonsten in etwa 5 Klassen dupliziert wäre, zentral an einer Stelle zusammengefasst werden. Des Weiteren hatte die `BMOSet`-Klasse die Möglichkeit, ihre Elemente nicht als eine Menge, sondern als eine Menge von Mengen zu speichern. Nachdem diese (meines Erachtens überflüssige) Option einerseits nirgends genutzt wurde, andererseits ihre Implementierung aber etliche Fehler aufwies, habe ich diese Funktionalität aus der Klasse entfernt.

Ein anderes Problem stellen die Exceptions dar. Die Basisklassen definieren eine eigene Exception-Hierarchie im `preference.exception`-Package, die von den einzelnen Implementierungen benutzt wird. Diese Exceptions werden dabei mit einem String initialisiert. Allerdings erzeugt der XPath-Parser häufig Exception, die zusätzlich zur „Nachricht“ noch weitere Unterexceptions enthalten. Diese würden bei der Instanziierung einer dieser benutzerdefinierten Exceptions verloren gehen. Um diese Daten beizubehalten wurde ein weiterer Konstruktor eingefügt, der eine Exception als Parameter erhält und diese als *Cause* benutzt – d. h. im Stacktrace tauchen dann auch alle Unterexceptions dieser Exception auf.

Wie oben bereits beschrieben musste die `MultiSetPreference` beim Auswerten ihrer Basispräferenz bisher immer den Attributselektor austauschen. Dieser Austausch

musste beim Serialisieren nach XML und beim Klonen wieder rückgängig gemacht werden. Diese Umwege entfallen nun, indem die **BasePreference**-Klasse (von der sowohl die **MultiSetPreference** als auch alle Basispräferenzen abgeleitet sind) Template-Implementationen für ihre Methoden **compare** und **level** erhielt, die den Attributselektor auf ihre Argumente anwenden und sich dann auf die abstrakten Methoden **compareEvaluated** bzw. **levelEvaluated** abstützen. Dadurch kann besagte **MultiSetPreference** (über eine neue Klasse **BasePreferenceBMOSet**, die die Vergleichsmethode von **BMOSet** überschreibt, um sich auf die neuen Methoden zu beziehen) direkt auf diese neuen Methoden zugreifen; andererseits wird die Aufgabe der Auswertung des Attributselektors aus den einzelnen Basispräferenzen entfernt, was den Code kürzer und besser wartbar macht. Dies hatte allerdings zur Folge, dass sämtliche Basispräferenzimplementierungen geringfügig angepasst werden mussten.

Bisher existierte für die **SCORE**-Präferenz, die jedem Wert über eine Scorefunktion eine reelle Zahl (den *Score*) zuweist und höhere Scores für besser erachtet, noch keine direkte Implementierung als Basispräferenz. Vielmehr stellte eine Score-Funktion einen Spezialfall einer Rank-Funktion dar (mit nur einem Attributselektor und keinen untergeordneten Score-Funktionen). Für die Implementierung in **Preference XPath** ist diese Form durchaus brauchbar; andere Projekte erforderten es aber, dass Score-Präferenzen als Basispräferenzen behandelt werden können (also Unterklassen von **BasePreference** sein sollen). Deshalb wurde eine weitere Klasse **ScorePreference** implementiert, die eine Score-Funktion in Form einer Basispräferenz bereitstellt.

Nachdem eine Basispräferenz immer genau einen Attributselektor hat, bekommt sie nun eine neue Methode **getAttributeSelector()**, die ebendiesen Attributselektor zurückgibt. Dies vereinfacht den Programmcode an einigen Stellen (vorher musste immer auf den ersten Eintrag des Attributselektoren-Arrays zugegriffen werden, verbunden mit dem Test, ob ein solcher überhaupt existiert).

Eine weitere Stelle der Codeduplizierung waren die Attributselektoren: Jeder Attributselektor muss sein Ergebnis in verschiedene Rückgabetypen konvertieren können (String, Zahl, Boolean). Dieser Code ist jetzt in eine abstrakte Klasse **AbstractAttributeSelector** ausgelagert, in der geeignete Templatemethoden für die Konvertierung enthalten sind.

Die **EXPLICIT**-Präferenz hatte des Weiteren einen Implementierungsfehler. Elemente, die nicht explizit spezifiziert sind, sollen gemäß [Kie02] schlechter als alle angegebenen Elemente sein. Bis hierhin entsprach die Implementierung noch der Spezifikation. Allerdings waren zwei Elemente, die beide nicht explizit spezifiziert waren, nicht unvergleichbar, sondern manchmal war das eine besser und manchmal das andere. Dies führte in Fällen, wo keines der spezifizierten Elemente in der Datenbank vorkam, zu falschen Ergebnismengen.

3.3 Serialisieren/Deserialisieren

Wie oben bereits beschrieben waren für die Serialisierung und Deserialisierung zwei eigene Klassen zuständig. Dies hatte den Nachteil, dass man beim Anpassen oder Neuschreiben von Präferenzklassen leicht vergessen konnte, auch diese beiden Klassen passend zu erweitern. Zudem ist die bei der Serialisierung nötige `instanceof`-Kaskade bei weitverzweigten Klassenbäumen dahingehend fehleranfällig, dass man Tests auf Unterklassen immer vor den Tests der Oberklassen einfügen muss, um den gewünschten Effekt (Unterklasse wird anders serialisiert als Oberklasse) zu erreichen. Diese Probleme lassen sich dadurch lösen, dass ein Teil des Serialisierungscode in die einzelnen Präferenzen ausgelagert wird; nun überprüft der Compiler, ob der Code vorhanden ist, und die Wahl der richtigen Implementierung für die jeweilige Präferenz erledigt sich (dank Polymorphie) von alleine. An die Deserialisierung muss man allerdings weiterhin selbst denken; wer aber Serialisierungscode schreiben muss, vergisst den Deserialisierungscode wohl nicht so leicht.

Um XML-spezifischen Code aus den Präferenzen herauszuhalten, sowie um dafür nötigen Code (wie etwa das Serialisieren einer Liste nach XML) nicht zu duplizieren, speichert jede Präferenz nun ihre Eigenschaften in einem `PreferenceAttributes`-Objekt. Dieses Objekt wird dann nach XML serialisiert. Für jeden Eigenschaftstyp (String, Liste, Tupel, SV-Relation) existiert eine eigene innere Klasse, die den jeweiligen Typ serialisieren kann. Als kleinen Nebeneffekt erlauben diese Attribute, Präferenzen auf einfache Weise auf Gleichheit zu überprüfen (sowie Hashcodes zu berechnen). Präferenzen, die auf dieselbe Weise erzeugt wurden (sowie Klone von Präferenzen) sind dabei immer gleich; es kann jedoch sein, dass zwei Präferenzen, die sich exakt gleich verhalten, nicht als gleich angesehen werden, weil sie über verschiedene Wege erzeugt wurden. Dieses Verhalten ist zwar unerwünscht; nachdem aber einige Stellen der Java-API (beispielsweise das `Comparator`-Interface) sich ebenso verhalten, ist es für einen Java-Programmierer nicht weiter verwunderlich.

Nachdem bei der Deserialisierung noch keine Instanz der jeweiligen Präferenzklasse existieren muss – ja die Klasse muss noch nicht einmal geladen sein – kann die Deserialisierung nicht direkt in die Präferenzklasse gepackt werden. Eine *Chain of Responsibility* (vgl. [GHJV94]), in der für jeden Präferenztyp ein eigener *Handler* eingetragen wird, wäre theoretisch denkbar, führt aber zu einer Klassenexplosion (pro Präferenztyp eine weitere Handlerklasse) und zu zusätzlichem Code, der die Kette initialisieren muss. Daher wurde der Deserialisierungscode im Wesentlichen unverändert belassen. Klassen, die den Extensions-Mechanismus verwenden, können ohnehin schon deserialisiert werden (die Serialisierung von Extension-Präferenzen erzeugt ein `<EXTENSION>`-Element, das den Klassennamen als Attribut enthält). Für eine potenzielle weitere Präferenz-Implementierung (also nicht XPath oder SQL), die aus irgendeinem Grunde eigene

Präferenztypen benötigt, unterstützt die Deserialisierungs-klasse nun zusätzlich zum eingebauten Deserialisierungscode, dass der Benutzer weitere Handler hinzufügt, die der Reihe nach aufgerufen werden.

Während der Anpassung der Serialisierung fiel auf, dass bei einigen Klassen – wie vorausgesehen – die Serialisierung und/oder Deserialisierung einfach vergessen wurde. Bei anderen Klassen wurden einige Attribute nicht mitserialisiert (wenn die Attribute und der Serialisierungscode in derselben Klasse stehen fällt das eher auf). Des Weiteren waren einige Schreib- oder Sinnfehler in den XML-Tag-Namen teilweise zwar bei der Deserialisierung (und im existierenden Repository), nicht aber bei der Serialisierung korrigiert – mit der Folge, dass sich frisch serialisierte Präferenzen häufig nicht mehr deserialisieren ließen. Diese Inkonsistenzen wurden dahingehend behoben, dass der Serialisierungscode nun immer die „richtigen Versionen“ benutzt; der Deserialisierungscode ist, wo es Sinn macht, abwärtskompatibel und kann damit auch bestehende Repositories problemlos deserialisieren.

Für das Preference-Repository existierte eine DTD, die allerdings nie zur Validierung benutzt wurde und daher auch noch auf dem Stand von 2002 war. Nachdem das neue Testframework gegen die DTD validiert, wurde die DTD als Nebeneffekt auch noch an die momentane Situation (zumindest soweit sie von Testfällen abgedeckt wird) angepasst.

3.4 LAYERED-Präferenz

Die bisherigen Basisklassen unterstützten keine LAYERED-Präferenz. Diese wurde jetzt zu den Basispräferenzen hinzugefügt, da man für die Darstellung von kategoriellen Präferenzen gerne darauf zurückgreifen würde.

Die `LayeredPreference`-Klasse enthält zusätzlich zum obligatorischen Attributselektor (und der SV-Relation, siehe Kapitel 3.5) ein Array aus Mengen von Strings. Die Elemente jeder dieser Mengen sind besser als die Elemente der nachfolgenden Mengen. Genau eine dieser Mengen ist `null` und repräsentiert alle nicht explizit aufgeführten Stringwerte – also `others`. Wie bereits in Kapitel 2.1.1 genannt, können mittels Layered-Präferenz alle anderen kategoriellen Präferenzen – außer der LAYERED-Präferenz – dargestellt werden. Die Klasse `StringSetPreference`, die diese anderen kategoriellen Präferenzen definierte, wurde deshalb so umgeschrieben, dass sie eine Unterklasse der LAYERED-Präferenz ist, sowie als *deprecated* erklärt. (Dies hat keinen Einfluss auf die Verfügbarkeit der einfachen Präferenzen in den Sprachen Preference SQL und Preference XPath. Die Parser dieser Sprachen, die bisher für einfache POS- oder POS/NEG-Präferenzen `StringSetPreference`-Objekte angelegt haben, werden nun stattdessen `LayeredPreference`-Objekte anlegen.)

3.5 D-Parameter und SV-Relationen

Wie oben bereits erwähnt, unterstützten die Basisklassen von sich aus keinerlei SV-Relationen oder *d*-Parameter. Für einige Präferenzen existierten bereits Extension-Klassen, die dies ermöglichen; diese haben allerdings eine unübersichtliche Syntax (die im Wesentlichen aus dem Klassennamen und einem String besteht, der von der Extension geparkt wird), die wenig mit der Syntax der Basisklassenpräferenzen gemeinsam hat. Andere präferenzbasierte Sprachen als Preference XPath (insbesondere Preference SQL) unterstützen im Moment noch gar keine Extensions, weshalb man dort bisher gar keine SV-Relationen oder *d*-Parameter benutzen kann.

Zudem konnten nur die triviale und die reguläre SV-Relation verwendet werden. Auch wenn diese in der Praxis üblicherweise ausreichen (normal reichten auch die implementierten Spezialfälle der LAYERED-Präferenz aus), will man möglicherweise ab und zu doch andere, explizite SV-Relationen benutzen.

Deshalb wurden sämtliche Basispräferenzen um eine SV-Relation erweitert; alle numerischen Basispräferenzen unterstützen jetzt zusätzlich einen *d*-Parameter. Bei zusammengesetzten Präferenzen wurde sichergestellt, dass diese den **SUBSTITUTABLE**-Zustand (der bei trivialer SV-Relation nicht auftreten kann) passend weitergeben.

SV-Relationen werden durch Objekte dargestellt, die das **SVRelation**-Interface implementieren:

```
public interface SVRelation {
    public abstract PreferenceAttributes getAttributes();
    public abstract boolean isSubstitutable(Object a, Object b);
}
```

Die erste Methode wird für das Serialisieren der SV-Relationen benutzt und ist für die Funktion der SV-Relation selbst irrelevant; wird eine Basispräferenz serialisiert, so werden die Attribute ihrer SV-Relation in die Präferenzattribute aufgenommen und damit mitserialisiert.

Die eigentliche Funktion verbirgt sich hinter der zweiten Methode: Für beliebige unvergleichbare Objekte *A* und *B* bestimmt ihr Rückgabewert, ob diese Objekte ersetzbar sind oder nicht. Die Methode wird (per Definition) niemals mit zwei vergleichbaren Elementen als Parameter aufgerufen. Dieser Kontrakt hat den Vorteil, dass die triviale und reguläre SV-Relation mit diesem Interface sehr einfach zu implementieren sind: die triviale SV-Relation liefert immer **false**, die reguläre immer **true**. Nachdem diese SV-Relationen sich bei jeder Präferenz gleich verhalten können, sind sie als Singletons implementiert.

Die dritte verfügbare SV-Relation ist die explizite SV-Relation. Diese erfordert in ihrem Konstruktor ein Array von ersetzbaren Mengen (also Äquivalenzklassen), sowie ein Flag (`others`), das angibt, ob alle nicht aufgeführten Elemente untereinander ersetzbar sein sollen. Ähnlich wie die EXPLICIT-Präferenz alle denkbaren endlichen Präferenzen abbilden kann, kann die explizite SV-Relation alle denkbaren endlichen SV-Relationen abbilden. Die triviale SV-Relation lässt sich als `SV ()` darstellen, die reguläre als `SV (others)`. Eine kompliziertere SV-Relation, bei der alle Farben außer rot, grün und blau ersetzbar sein sollen, rot aber nur mit grün und blau gar nicht, lässt sich als `SV (('rot', 'gruen'), ('blau'), others)` modellieren.

Die Syntax der expliziten SV-Relation sieht also ähnlich aus wie die der LAYERED-Präferenz. Ein wesentlicher Unterschied in der Semantik besteht darin, dass bei der LAYERED-Präferenz die Reihenfolge der Mengen relevant ist, bei der expliziten SV-Relation hingegen nicht.

Weitere, komplexere SV-Relationen für Spezialanwendungen können mit diesem Interface leicht konstruiert werden. Nachdem die explizite SV-Relation für numerische Präferenzen eher ungeeignet ist, wäre es beispielsweise denkbar, eine intervallbasierte SV-Relation, die Elemente innerhalb eines Intervalls als ersetzbar ansieht, zu entwickeln.

Ein anderes Beispiel für eine spezielle SV-Relation ist die `LevelBasedSVRelation`, die alle Elemente, die bezüglich einer Basispräferenz auf derselben Ebene liegen, als ersetzbar ansieht. Hierbei kann die Basispräferenz, die für die Bestimmung des Levels dient, durchaus von der Präferenz, für die die SV-Relation verwendet wird, verschieden sein.

3.6 Erweiterungsklassen

Nachdem nun sämtliche bisher vorhandenen Extensionfunktionen direkt von Preference XPath unterstützt werden, wurden sie entfernt. Als Beispielextension wurde eine neue Präferenz angelegt, die einerseits die verfügbare Schnittstelle nahezu komplett ausnutzt, deren Anwendung andererseits aber zu unrealistisch ist, um sie jemals in die Basisklassen aufzunehmen: Die Erpresserbriefschreiber-Präferenz.

Angenommen, jemand möchte einen Erpresserbrief schreiben (oder möglicherweise mehrere). Da seine Handschrift dem Erpressten möglicherweise bekannt ist, möchte er seine Texte aus Buchstaben (oder alternativ Wörtern) alter Zeitungsüberschriften zusammenkleben. Nachdem er nun eine größere Anzahl Überschriften gesammelt hat, geht ihm der Platz aus und er überlegt, welche der Überschriften er wegwerfen könnte. Er möchte natürlich genau diejenigen Überschriften behalten, die für seinen Zweck am nützlichsten sind. Glücklicherweise verfügt unser Erpresser über einen Fotokopierer. Er kann daher alle Überschriften wegwerfen, deren Buchstaben (bzw. Wörter) in anderen Überschriften komplett enthalten sind. Soweit das (zugegebenermaßen unrealistische) Beispiel. Mathematisch gesehen reduziert sich dieses Beispiel auf elementare Mengenlehre: Ist eine Buchstabenmenge (bzw. Wortmenge) eine Teilmenge einer anderen, so ist die Obermenge besser als die Teilmenge. Überschriften, die dieselben Buchstaben (bzw. Wörter) enthalten, sind ersetzbar; die SV-Relation ist in dieser Präferenz also sozusagen schon eingebaut. Parameter der Präferenz dienen vielmehr dazu, einzustellen, wie der Erpresser die Wörter zerlegt – buchstabenweise oder wortweise –, ob er Wert auf Groß-/Kleinschreibung legt und ob er Wörter verkürzen will (beispielsweise ein „diese“ zu einem „die“).

Diese Extension ist großzügig dokumentiert und deshalb als Vorlage für die Entwicklung eigener Erweiterungen bestens geeignet.

4 Anpassung an JDK1.5

Die Hauptaufgabe dieser Diplomarbeit bestand darin, Preference XPath unter Java 1.5 lauffähig zu bekommen. Zusätzlich soll die Kompatibilität zu Java 1.4.1 und 1.4.2 erhalten bleiben, da viele Produktsysteme heute noch diese Laufzeitumgebungen verwenden.

4.1 Probleme und Lösungsmöglichkeiten

Wenn man die API von Xalan und diejenige des `javax.xml.xpath`-Pakets vergleicht, fällt letztere API viel schmäler aus. Dies ist verständlich – schließlich soll die API mit jedem XPath-Parser zusammenarbeiten können. Die offizielle API beschränkt sich darauf, benutzerdefinierte Namespaces, Funktionen und Variablen definieren zu können, sowie eine XPath-Anfrage zu compilieren und die compilierte Anfrage auf einem DOM-Element auszuführen.

Theoretisch reichen diese Funktionen für Preference XPath problemlos aus. In der Praxis ergeben sich allerdings einige Hürden: Einerseits erfordert die API, bereits beim Ausführen den erwarteten Ergebnistypen anzugeben (also *NodeSet*, *String*, *Boolean* oder *Number*). Kann der tatsächliche Ergebnistyp nicht in diesen konvertiert werden, erhält man eine nichtssagende Exception. Ansonsten verliert man durch die implizite Konvertierung Daten. Demgegenüber liefert die Xalan-API ein `XObject`-Objekt zurück, das weiß, welchen Typs es ursprünglich war. Gerade bei der Auswertung „innerer Anfragen“, die vom Attributselektor ausgehen und für jedes Element eines Zwischenergebnisses ausgewertet werden müssen, sind Zahlen oder Strings nicht so selten wie sie es bei der eigentlichen Anfrage sind. Der Attributselektor weiß zwar, welcher Ergebnistyp (Zahl oder Text) gefordert ist; allerdings kann durch die Mengensemantik anstelle einer Zahl auch eine Liste von Zahlen als Ergebnis auftauchen; eine Konvertierung der Liste in eine Zahl vernachlässigt jedoch alle bis auf das erste Element. Verlangt man grundsätzlich nach einer Liste, treten Probleme auf, wenn normale XPath-Funktionen wie `count` in der Unterabfrage vorkommen; diese internen Funktionen können nämlich keine Liste als Ergebnis liefern.

Dieses Problem wird dadurch umgangen, dass die XPath-Anfrage vor der Auswertung rudimentär syntaktisch untersucht wird, um den Ergebnistyp abzuleiten – steht an oberster Ebene eine vordefinierte Funktion, deren Ergebnistyp bekannt ist, so kann dieser benutzt werden. Kann der Ergebnistyp auf diese Weise nicht bestimmt werden, so wird auf Verdacht der Ergebnistyp *NodeSet* ausprobiert. Schlägt dies fehl, dann wird der Ergebnistyp *String* benutzt. In einen dieser beiden Typen kann jedes Ergebnis verlustfrei konvertiert werden. Ist der Typ einer Anfrage einmal bestimmt, so wird dieser gecached, um ihn bei weiteren Anfragen (die im Fall eines Attributselektors fast immer auftreten) wiederzuverwenden. Benötigt die jeweilige Präferenz als Ergebnis einen anderen Typ als den Typ, in dem der Wert vorliegt, so wird der Wert nicht über die vorhandenen Typkonvertierfunktionen der API konvertiert, sondern händisch (wobei die Implementierung sich an [W3C99] orientiert), weil die Typkonvertierung über die API extrem langsam erfolgt.

Ein weiteres Problem besteht darin, dass bei jeder Unteranfrage das gesamte XML-Dokument von DOM nach DTM konvertiert werden muss (schließlich kann sich ein XPath-Ausdruck auch auf Knoten außerhalb des Kontextknotens beziehen); dies kann bei größeren Dokumenten mit vielen Datensätzen einen deutlich spürbaren Overhead verursachen. Dieses Problem könnte dadurch entschärft werden, dass bei Unteranfragen, die nur Daten innerhalb des Kontextknotens benötigen, dieser Knoten vor der Auswertung in ein neues DOM-Dokument kopiert wird – der XML-Ast wird vom Dokument sozusagen abgesägt. Das große Problem hierbei besteht darin, zu erkennen, ob die Unteranfrage wirklich nur innerhalb des Kontextknotens abläuft.

Und selbst wenn diese Art des *Prunings* implementiert ist, läuft die direkte Implementierung noch deutlich langsamer als es die JDK1.4.1-Implementierung tat.

Des Weiteren soll es bekanntlich auch möglich sein, das Programm weiterhin unter JDK1.4.1 oder 1.4.2 auszuführen. Hierbei ist zu bemerken, dass sich die Schnittstelle für benutzerdefinierte Funktionen von 1.4.1 auf 1.4.2 komplett geändert hat. Die Verwendung desselben Codes für beide Versionen ist also aussichtslos. Außerdem verträgt sich – wie oben bereits erwähnt – die dynamische Auswertung unter JDK1.4.2 nicht mit den ThreadLocals.

Die Verwendung von Xalan-spezifischen Klassen (wie *XObject*, *XPathContext* und *PrefixResolver*) in der externen API von Preference XPath konnte nicht beibehalten werden – die offizielle API enthält diese Klassen ohnehin nicht, in der inoffiziellen API sind sie von Java 1.4 auf 1.5 aus dem Paket `org.apache.xpath` nach `com.sun.org.apache.xpath.internal` umgezogen. Deshalb musste die externe API geändert werden, was leider auch Veränderungen in jedem Programm, das Preference XPath verwendet, erfordert.

4.2 Die Adapterschnittstelle

Um diese Probleme so gut wie möglich in den Griff zu bekommen, wurden alle JDK-spezifischen Funktionen hinter einer Adapterschnittstelle gekapselt (vgl. [GHJV94]). Dies verhindert zwar nicht, dass für jedes JDK spezifischer Code vorhanden ist (der auch mit dem jeweiligen dazu passenden Compiler kompiliert werden muss), hilft aber dabei, diesen Code möglichst kompakt zu halten sowie zur Laufzeit die passende Implementierung auszuwählen. Letzteres erlaubt es, ein- und dasselbe Programm sowohl auf JDK1.4 als auch auf JDK1.5 auszuführen.

Um die Adapterklassen und -interfaces deutlich sowohl von den restlichen Preference-XPath-Klassen als auch von den Klassen der spezifischen XPath-Implementierung abzugrenzen, liegen erstere im `preference.xpath.adapter`-Paket. Zudem beginnen ihre Namen alle mit PXA für Preference-XPath-Adapter.

Alle anderen Klassen von Preference XPath (außer den Adapterimplementationen in `preference.xpath.adapter.*`) sind nun unabhängig vom XML-Parser und können mit jedem JDK (ab 1.4) kompiliert werden.

Die einzige richtige Klasse in diesem Adapterpaket heißt `PXAFactory`. Ansonsten besteht das Paket noch aus 5 Interfaces (`PXAResolver`, `PXAXPath`, `PXAXPathDTM`, `PXAPreferenceXPath` und `PXA\XObject`) sowie einer Exception (`PXAWrappedException`).

Die Factoryklasse weist folgende Schnittstelle auf:

```
package preference.xpath.adapter;

public abstract class PXAFactory {

    public abstract PXAResolver createDefaultResolver
        (Node xPathExpressionContext);

    public abstract PXAResolver createExtensionResolver
        (PXAResolver baseResolver, String prefix, String namespace,
         UserDefinedFunctionResolver functionResolver);

    public abstract PXAXPath createXPath(String exprString,
        PXAResolver resolver) throws PXAWrappedException;

    public abstract PXAPreferenceXPath createPreferenceXPath
        (String exprString, PreferenceContext pc,
         boolean useMultiDocuments) throws PXAWrappedException;
```

```
public abstract PXAXObject createXNodeSetFromDOMNode(Node node,
    PXAResolver resolver);

public static void preloadInstance(PXAFactory factory) { /*...*/ }

public static PXAFactory getInstance() { /*...*/ }
}
```

Diese Klasse ist als *Singleton* (vgl. [GHJV94]) implementiert. Vor dem ersten Aufruf von `getInstance()` kann über die `preloadInstance()`-Methode zur Laufzeit eine bestimmte `PXAFactory`-Implementierung registriert werden. Verwendet man die `preloadInstance`-Methode nicht, so wird beim ersten Aufruf von `getInstance()` dynamisch eine Factory gesucht und instanziiert.

Diese Suche läuft wie folgt ab: Ist die Systemproperty `preference.xpath.adapter.factory` gesetzt, so wird die Klasse, deren Klassenname in der Property steht, als Factory benutzt. Anderenfalls gibt es die Möglichkeit, zur Compilezeit der `PXAFactory`-Klasse (per Konstante) eine bestimmte Factory festzulegen. Ist dies auch nicht erfolgt, so wird geprüft, ob für die jeweilige Java-Version eine spezielle Factory vorliegt (die Lookup-Tabelle hierfür ist zur Compilezeit festgelegt). Zu guter Letzt wird dann, wenn all dies erfolglos ist, die Klasse `preference.xpath.adapter.documented.PXAFactoryDA`, die lediglich die dokumentierte API von Java 1.5 oder höher benutzt, instanziiert. Schlägt auch deren Instanziierung fehl, wird ein Fehler geworfen.

Das `PXAResolver`-Interface ist ein *Token*-Interface – ein Objekt, das dieses implementiert, kann XPath-Funktionen für den jeweiligen Adapter auflösen. Wie bei Token-Interfaces üblich sind diese nicht zwischen verschiedenen Adapterimplementationen kompatibel. Nachdem aber in der Regel ohnehin nur ein Adapter gleichzeitig verwendet wird, stellt dies kein Problem dar. Da jede Preference-XPath-Anfrage einen eigenen Klon dieses Objekts (über die einzige öffentliche Methode des Interfaces) anlegt, sind die implementierenden Klassen weiterhin dafür geeignet, implementationsspezifische Daten von der Hauptanfrage in die Unteranfragen zu retten – der „Trick“ mit den *ThreadLocals* ist also nicht länger nötig.

Hier findet sich auch der einzige Nachteil dieses Ansatzes: Die `PXAResolver`-Implementationen müssen für jede neue Präferenzanfrage geklont werden. Dies stellt aber – insbesondere da diese Objekte sonst keinen Status haben und daher billig geklont werden können – kein größeres Problem dar.

Durch Aufruf von `createDefaultResolver` kann ein Resolver, der nur die Standard-XPath-Funktionen und die präferenzspezifischen Funktionen auflösen kann, angelegt werden. Die Bedeutung von `createExtensionResolver` wird in Kapitel 4.4 erklärt.

Die Methode `createPreferenceXPath` wird vom Präferenzcompiler aus aufgerufen und erzeugt aus dem `PreferenceContext` (der das `Preference`- und das `Locpath`-Array enthält) und der umgeschriebenen XPath-Anfrage das eigentliche XPath-Objekt. Ist das `Locpath`-Array im `Preference`-Kontext nichtleer, kann die Anfrage auf mehreren Dokumenten gleichzeitig ausgeführt werden – in diesem Fall muss `useMultiDocuments` wahr sein. (Es ist nun auch möglich, in derselben Java-Instanz beide Modi gemischt zu verwenden. Vorher wurde dies durch statische Attribute verhindert). Die Hauptanwendung für den `MultiDocument`-Modus ist Apache Xindice; er kann aber auch für andere Dokumente benutzt werden.

Mittels `createXPath` werden die XPath-Unterausdrücke vom Attributselektor erzeugt. Diese dürfen entgegen des Methodennamens auch Präferenz-Anfragen enthalten; der Unterschied zur zuvor beschriebenen Methode besteht primär darin, dass die Unterausdrücke den Präferenzkontext (sowie den Rest der Informationen, die im `PXAResolver` gespeichert sind) des Hauptausdrucks übernehmen. Zudem unterstützen diese Anfragen keinen `MultiDocumentMode` mehr – nachdem die Unteranfragen auf jedem Ergebnisknoten einzeln ausgeführt werden, ist es dort ohnehin nicht mehr nötig.

Die `execute`-Methoden des `PXAPreferenceXPath`- und `PXAXPath`-Interfaces erfordern, wie bereits in Kapitel 4.1 erklärt, einen Ergebnistyp als Parameter. Ist der Ergebnistyp unbekannt, kann stattdessen `PXAXPath.UNKNOWN_RESULT` angegeben werden, was je nach Adapter die Abfrage ausbremst.

Das Ergebnis einer Anfrage wird in ein `PXAXObject` eingepackt, dessen Interface der Schnittstelle der Xalan-Klasse `XObject` nachempfunden ist. Es existieren also Methoden, um das Ergebnis in jeden von XPath unterstützten Ergebnistyp (*Nodeset*, *String*, *Number*, *Boolean*) umzuwandeln. Nachdem außer *Nodeset* kein anderer Typ zu einem *Nodeset* konvertiert werden kann, aber jeder Typ nach *Boolean*, *Number* oder *String*, existiert eine weitere Methode, mit der getestet werden kann, ob das Ergebnis ein `NodeSet` ist.

Treten bei der Auswertung parserspezifische Exceptions auf, so werden diese in `PXAWrappedException` eingepackt und weitergeworfen. Diese Exceptionklasse ist eine Unterklasse von `PreferenceException`; dies vereinfacht die Benutzung der API, da einerseits nur eine Art von Exceptions gefangen werden muss, andererseits durch spezifisches Fangen bestimmter Unterklassen flexibel auf die verschiedenen Fehler reagiert werden kann.

Nachdem der Attributselektor jetzt nicht nur mit DTM-Knoten, sondern auch mit DOM-Knoten umgehen können muss, wurde die alte Klasse `PrefDTMNode` durch eine Hierarchie ersetzt, die aus dem Interface `PrefNode` (das einen allgemeinen Knoten eines XML-Dokuments beschreibt) und aus spezifischen Unterklassen für DTM und DOM besteht. Nachdem die XPath-Unterabfragen die meiste Rechenzeit verbrauchen,

ist ein Caching-Konzept, das einmal abgefragte Werte für die weiteren Vergleiche zwischenspeichert, unerlässlich. Ein solches Konzept existierte schon, war aber ineffizient programmiert (duplizierter Code) und konnte nicht so einfach für DOM-Knoten benutzt werden. Dieses Caching wurde vereinfacht und dahingehend umgeschrieben, dass es auch mit DOM-Knoten funktioniert. Des Weiteren unterscheidet die neue Klasse zwischen veränderbaren und unveränderbaren DTM-Knoten. Veränderbare Knoten werden für schnelle Tests bei HashMaps benutzt, unveränderbare in der HashMap selbst abgelegt (Bisher wurden die selben Knoten für beides verwendet; man musste immer darauf achten, dass ein Knoten, der bereits in einer HashMap lag, nicht mehr verändert wurde).

Die ursprüngliche Implementierung erzeugte eine Unterklasse der XPath-Klasse, die für die Weitergabe der Parameter an die XPath-Implementierung zuständig war. Dies hatte den unerwünschten Nebeneffekt, dass interne Aufrufe der Methoden dieser Klasse (aus dem Xalan-Framework heraus) auch die überschriebenen Methoden aufrufen. Nachdem diese Art der Implementierung bei XPath-APIs, die nur aus Interfaces bestehen (wie der neuen XPath-API von Java 1.5) ohnehin nicht möglich sind, und der eigentliche Vorteil (Programme „merken nicht“, dass sie es mit Preference XPath zu tun haben) durch den Adapter ohnehin nicht mehr besteht, wird nun in allen Implementationen Delegation anstelle von Vererbung verwendet. Alle Delegatorobjekte implementieren das `PXAXPath`-Interface; Delegatoren für DTM-fähige XPath-Parser dessen Unterinterface `PXAXPathDTM`. Letzteres Interface bietet eine Methode, die für `PrefDTMNodes` nötig ist. Delegatorobjekte für präferenzbasierte Anfragen (die einen eigenen Präferenzkontext besitzen) implementieren das Interface `PXAPreferenceXPath`, welches das Ausführen einer Anfrage auf mehr als einem Kontextknoten (*MultiDocumentMode*) unterstützt.

4.3 Die Fassadenklassen

In der alten Version sah der Code für eine typische Preference-XPath-Anfrage folgendermaßen aus:

```
import org.apache.xml.utils.PrefixResolverDefault;
import org.apache.xpath.XPath;
import org.apache.xpath.XPathContext;
import org.apache.xpath.objects.XObject;

import javax.xml.transform.TransformerException;
import org.w3c.dom.*;

import preference.engine.DefaultDOMDocument;
import preference.exception.PreferenceException;
import preference.xpath.PreferenceCompiler;
import preference.xpath.xerces.DOMEngine;

public class ExampleAlteAPI {

    public static NodeList executeQuery(Document doc, String query)
        throws PreferenceException, TransformerException {
        DOMEngine engine = new DOMEngine(new DefaultDOMDocument());
        XPathContext context = new XPathContext();
        XPath xpath = PreferenceCompiler.parsePXPath(engine, query, null,
            new PrefixResolverDefault(doc.getDocumentElement()),
            XPath.SELECT);
        XObject xObj = xpath.execute(context, doc.getDocumentElement(),
            new PrefixResolverDefault(doc.getDocumentElement()));
        if (xObj.getType() == XObject.CLASS_NODESET) {
            return xObj.nodelist();
        } else {
            throw new PreferenceException("Result is no NodeList");
        }
    }
}
```

Wie man hier recht gut sehen kann, musste man zur Verwendung von Preference XPath Xalan-interne Klassen benutzen. Nachdem diese in Java 1.5 nicht mehr (in demselben Package) zur Verfügung stehen, musste die Preference-XPath-API notgedrungen geändert werden.

Zum Vergleich: Bei Verwendung der Adapterklassen sieht der Code folgendermaßen aus:

```
import preference.xpath.adapter.*;

import javax.xml.transform.TransformerException;
import org.w3c.dom.*;

import preference.engine.DefaultDOMDocument;
import preference.exception.PreferenceException;
import preference.xpath.PreferenceCompiler;
import preference.xpath.DOMEngine;

public class ExampleNeueAPI {

    public static NodeList executeQuery(Node node, String query)
        throws PreferenceException, TransformerException {
        DOMEngine engine = new DOMEngine(new DefaultDOMDocument());
        PXAFactory factory = PXAFactory.getInstance();
        PXAResolver resolver = factory.createDefaultResolver(node);
        PXAXPath xpath = PreferenceCompiler.parsePXPath(engine, query,
            resolver, false);
        PXAXObject xObj = xpath.execute(node, PXAXPath.UNKNOWN_RESULT);
        if (xObj.isNodeSet()) {
            return xObj.nodelist();
        } else {
            throw new PreferenceException("Result is no NodeList");
        }
    }
}
```

Hier erkennt man gut, dass die neue API versucht, die alte so gut es geht zu imitieren. Dies erlaubt es, Programme mit minimalem Aufwand auf die neue API zu portieren. Andererseits fällt auch auf, dass für eine so grundlegende Funktion wie das Auswerten einer Query umfangreicher Java-Code erforderlich ist, der auf viele Teile der Preference-XPath-API zurückgreift. Dies hat zur Folge, dass jedes Clientprogramm recht stark mit der API *gekoppelt* ist. Kopplung erhöht Abhängigkeiten und verringert die Portabilität – die nächste Änderung an XPath würde also möglicherweise schon wieder eine API-Änderung nötig machen.

Eine übliche Lösung für dieses Problem ist das *Fassaden-Pattern* (siehe [GHJV94]). Eine Fassadenklasse kapselt die am häufigsten verwendeten Funktionen, so dass die meisten Clientprogramme im Idealfall nur noch mit der Fassadenklasse gekoppelt sind.

Die dadurch entstehende schmale API ist auch viel leichter stabil zu halten als die Unmenge an öffentlichen Methoden, die die Preference-API sonst noch zu bieten hat.

Die Fassadenklasse befindet sich im `preference.xpath.adapter.facade`-Paket neben zwei weiteren kleineren Utilityklassen, die Debugging- und XML-Funktionen zur Verfügung stellen. Sie bieten im Endeffekt alles, was die Beispiel-Anwendung benötigt.

Mit Verwendung der Fassadenklasse sieht unser Beispielcode folgendermaßen aus:

```
import preference.xpath.adapter.*;

import javax.xml.transform.TransformerException;
import org.w3c.dom.*;

import preference.exception.PreferenceException;
import preference.xpath.facade.PreferenceXPathFacade;

public class ExampleFacade {

    public static NodeList executeQuery(Node node, String query)
        throws PreferenceException, TransformerException {
        PreferenceXPathFacade facade = new PreferenceXPathFacade();
        PXAXObject xObj = facade.executePreferenceXPath(query, node,
            PXAXPath.UNKNOWN_RESULT, false);
        if (xObj.isNodeSet()) {
            return xObj.nodelist();
        } else {
            throw new PreferenceException("Result is no NodeList");
        }
    }
}
```

Außer der Instanziierung der Fassade (was am Anfang des Programmes erfolgen kann) ist nun nur noch ein Aufruf nötig, um eine Anfrage auszuführen. Die Fassade bietet zusätzlich eine weitere Methode an, die unter der Annahme, dass das Ergebnis immer ein *NodeSet* sein wird, auch die letzten 5 Codezeilen noch einspart.

Der Vollständigkeit halber ist die komplette Schnittstelle der Fassadenklasse auf der nächsten Seite abgedruckt.


```
public class PreferenceXPathFacade {

    public PreferenceXPathFacade(IDOMRepositoryDocument repository)
        { /* ... */ }
    public PreferenceXPathFacade(File repositoryFile) { /* ... */ }
    public PreferenceXPathFacade() { /* ... */ }

    public PXAXObject executePreferenceXPath(String xPath, Node contextNode,
        int returnType, boolean useMultiDocuments)
        throws PreferenceException { /* ... */ }

    public NodeList executePreferenceXPath(String xPath, Node contextNode)
        throws PreferenceException { /* ... */ }

    public NodeList executePreferenceXPath(String xPath,
        Node[] contextNodes) throws PreferenceException { /* ... */ }

    public PXAXObject executePreferenceXPath(String xPath,
        Node[] contextNodes, int returnType)
        throws PreferenceException { /* ... */ }

    public DOMEngine getDOMEngine() { /* ... */ }

    public static Class classForFile(File file) { /* ... */ }

    public void registerRankFunctionClass(Class rankClass)
        throws PreferenceException, InstantiationException,
        IllegalAccessException { /* ... */ }

    public void registerRankFunction(CombiningFunction rankFunction)
        throws PreferenceException { /* ... */ }

    public void registerScoreFunctionClass(Class scoreClass)
        throws InstantiationException, IllegalAccessException,
        PreferenceException { /* ... */ }

    public void registerScoreFunction(ScoreFunctionBase scoreFunction)
        throws PreferenceException { /* ... */ }

    public void registerExtensionXPathFunction(UserDefinedFunction func)
        { /* ... */ }

    public void useReflectionForExtensionXPathFunctions
        (boolean enabled) { /* ... */ }

}
```

Die Konstruktoren erlauben es, verschiedene Preference-Repositories zu verwenden. Der Standardkonstruktor verwendet ein leeres Repository. Die vier `executePreferenceXPath`-Methoden sind für die Auswertung von XPath-Anfragen gedacht – je nach gewünschtem Parameter- und Rückgabetyt steht für jeden Zweck eine Methode zur Verfügung. `getDOMEngine` erlaubt den Zugriff auf die XPath-Implementierung, um beispielsweise das Repository zu manipulieren oder Abfragen ohne Verwendung der Fassade auszuführen.

Die restlichen Methoden sind für die Registrierung von Score- und Rank-Funktionen sowie benutzerdefinierten XPath-Funktionen gedacht. Für Testzwecke besonders nützlich ist hierbei die `classFromFile`-Methode, die es erlaubt, aus einem (per Dateiauswahl-GUI gewählten) Pfad die jeweilige Klasse zu instanziiieren. Gerade wegen der Implementation von Paketen als Unterverzeichnissen ist dies keine triviale Angelegenheit.

Wie bereits erwähnt, enthält das `preference.xpath.facade`-Paket außerdem noch zwei Utilityklassen. Die erste (`XMLUtilities`) bietet Methoden zum Laden und Speichern von DOM-Dokumenten sowie zum Transfer von Knoten zwischen inkompatiblen DOM-Implementationen (wie sie beispielsweise bei Verwendung von Xindice auftreten).

Die zweite erlaubt es, eine Preference-XPath-Anfrage umzuschreiben und das Ergebnis, bestehend aus der neuen XPath-Anfrage, dem Preference-Array und dem Locpath-Array, in XML zu serialisieren. Dies erleichtert insbesondere das Debugging des XPath-Parsers. Da es auch beim Auftreten eines XPath-Fehlers auf einem Produktivsystem (was sich erfahrungsgemäß nicht hundertprozentig vermeiden lässt) nützlich sein kann, diese Informationen in eine Logdatei zu speichern, wurde diese Funktionalität, die ursprünglich direkt in der XPath-GUI implementiert war, in eine eigene Klasse `QueryRewriting` ausgelagert.

4.4 Selbstdefinierte XPath-Funktionen

Reines XPath ermöglicht es dem Anwender, den Sprachschatz bei Bedarf mit eigenen Funktionen zu erweitern. Diese Funktionalität wird von Preference XPath, wie bereits beschrieben, auch ausgiebig genutzt. Nichtsdestoweniger sollte es auch für den Anwender von Preference XPath weiterhin möglich sein, eigene Funktionen zu definieren.

Der Xalan-Parser von JDK 1.4.1 behandelt selbstdefinierte Funktionen an zwei Stellen – der `PrefixResolver` ist dafür zuständig, ein Präfix in einen Namespace zu übersetzen; dieser Namespace wird dann in der `ExtensionsTable` (die selbst im

`XPathContext` vorgehalten wird) nachgeschlagen, um die passende Funktion zu finden. Nachdem sowohl `XPathContext` als auch `PrefixResolver` (wie im Beispielcode aus Kapitel 4.3 ersichtlich war) vom Benutzer erzeugt wurden, konnte diese Möglichkeit dort genutzt werden.

Generell unterstützt auch die API von JDK 1.5 derartige selbstdefinierte Funktionen, allerdings über eine ganz andere Schnittstelle. Nachdem die neue Implementierung aber unabhängig vom JDK sein soll, kann auf derartige Implementierungsdetails nicht mehr direkt von der benutzenden Anwendung zugegriffen werden. Deshalb wurde die Adapterschnittstelle um eine parserunabhängige Möglichkeit erweitert, eigene XPath-Funktionen zu definieren. Diese müssen hierfür folgende Schnittstelle implementieren:

```
public interface UserDefinedFunction {
    public String getName();
    public int getArity();
    public int getReturnType();

    public Object evaluate(PXAXObject[] arguments)
        throws PreferenceException;
}
```

Die ersten drei Methoden geben Name, Stelligkeit und Rückgabetyt der Funktion zurück. Der Rückgabetyt wird benötigt, um den Ergebnistyp einer Subexpression, die als äußerste Funktion diese selbstdefinierte Funktion aufweist, bestimmen zu können – bei der offiziellen JDK1.5-API wird dieser ja, wie bereits angegeben, bei der Ausführung einer Anfrage benötigt. Als mögliche Rückgabetyten werden die Konstanten aus dem `PXAXPath`-Interface verwendet.

Die letzte Methode ist für die eigentliche Ausführung verantwortlich. Sie erhält als Parameter gekapselte `XObjects`, auf denen die Anfrage auszuführen ist. Damit kann die Funktion selbst bestimmen, in welchem Format sie die Parameter verwenden will. Der Rückgabewert wird dann als normales Java-Objekt, also als String, Double, Boolean oder NodeList, das natürlich kompatibel zum deklarierten Rückgabetyt sein muss, zurückgegeben.

Hat man nun eine oder mehrere solche Funktionen implementiert (einige Beispiele, die von Unit-Tests verwendet werden, finden sich im `preference.xpath.examples.extfunctions`-Paket), so kann über die `createExtensionResolver`-Methode der Adapterklasse ein Resolver registriert werden, der diese Funktionen kennt. Wird dieser Resolver für die Auswertung von Preference-XPath-Anfragen benutzt, so können dort diese Funktionen benutzt werden.

Prinzipiell können diese Resolver (als Unterklasse von `UserDefinedFunctionResolver`) selbst programmiert werden; für die häufigeren Fälle (Funktionen sind bereits in ein Array geladen bzw. Funktionen werden per Reflection gesucht) existieren bereits Standardklassen.

Benutzer der neuen Fassadenklasse müssen auf die Verwendung von Extension-Funktionen nicht verzichten: Diese Klasse erlaubt es ebenfalls, über `registerExtensionXPathFunction` eigene Funktionen der Fassade bekanntzumachen, die bei der weiteren Benutzung der Fassade verwendet werden können. Hierbei ist der Präfix für die neuen Funktionen immer „`extns`“, d. h. eine eigene Funktion `beispiel` wird in XPath als `extns:beispiel` angesprochen. Hat man die Benutzung von Reflection nicht deaktiviert, so können über den Namespace-Präfix „`reflns`“ auch Funktionen per Reflection nachgeladen werden; ist hierbei kein Paket angegeben, so wird die Klasse im Paket `preference.extensions` gesucht.

Hat man beispielsweise eine `extns:wordcount`-Funktion spezifiziert, die die Anzahl der Wörter eines Parameters zählt (diese Funktion wird auch im Testframework als Beispiel verwendet, siehe `preference.xpath.examples.extfunctions.WordCountFunction`), so kann man die Schüler mit den meisten Vornamen über folgende Anfrage ermitteln:

```
/db/schule/klasse/schueler#[extns:wordcount(@vorname) highest]#
```

4.5 Implementationen der Adapterschnittstelle

4.5.1 Adapter für JDK1.4.1/1.4.2

Nachdem die Preference-XPath-Klassen so umgeschrieben wurden, dass sie den neuen Adapter verwenden, wurde aus den nicht verwerteten Teilen des alten Codes (da Preference XPath weiterhin unter JDK1.4 funktionieren soll) zunächst ein Adapter für JDK1.4 implementiert.

Hierbei machte sich neben den bereits beschriebenen Problemen besonders bemerkbar, dass sich die Xalan-API von JDK1.4.1 auf JDK1.4.2 spürbar geändert hat. War unter JDK1.4.1 noch der generische `org.apache.xalan.ExtensionHandler` für die Behandlung eigener XPath-Funktionen zuständig, so gibt es seit JDK1.4.2 dafür eine Extraklasse `org.apache.xpath.ExtensionProvider`. Ersterer unterstützte es, Extensions direkt über Reflection zu laden, d. h. die Funktion `beispiel` aus dem Namespace `xalan://preference.xpath.Funktionen` würde nach einer öffentlichen Klassenmethode namens `beispiel` in der `preference.xpath.Funktionen`-Klasse suchen.

Diese im alten Code eingesetzte Technik wurde im neuen Code aus mehreren Gründen verworfen:

- Einerseits erlaubt sie keinerlei Checks zur Compilezeit, ob die Funktionen wirklich vorhanden sind.
- Des Weiteren werden Klassen, die nur über Reflection geladen werden, von modernen IDEs als „unreferenziert“ eingestuft, was dazu einlädt, die Klasse bei Gelegenheit zu löschen.
- Andererseits steht diese Möglichkeit außer unter JDK1.4.1 vermutlich nirgends zur Verfügung.
- Zu guter Letzt sind gerade diese statischen Methoden, die keinerlei Möglichkeit haben, auf irgendwelche Kontextinformationen zuzugreifen, schuld daran, dass die in Kapitel 4.1 beschriebenen ThreadLocals überhaupt nötig waren.

Genügend Gründe, um den Code ohne Verwendung dieser automatischen Funktionsuche zu implementieren.

Nachdem für JDK1.4.1 und JDK1.4.2 (wie auch für die in 4.5.5 beschriebene externe Lösung) verschiedene Schnittstellen benötigt werden, andererseits aber auch eine Menge Code übereinstimmt, wurde der übereinstimmende Teil im Package `preference.xpath.adapter.xalanbase` abgelegt. Die eigentliche Anbindung der Erweiterungen befindet sich dann in unterschiedlichen Packages, die auch jeweils eine eigene `PXAFactory`-Implementierung beinhalten.

Abgesehen von den Namen der Schnittstellen funktionieren aber alle diese Implementationen gleich: Bei jedem Erzeugen eines Preference-XPath-Objekts wird eine Kopie des momentanen ExtensionProviders angelegt, in dem der PreferenceContext vermerkt wird. Wird nun eine Präferenzfunktion aus diesem XPath-Ausdruck aufgerufen, so muss der ExtensionProvider diese ausführen; nachdem der Kontext genau dort gespeichert ist, sind keine ThreadLocals oder ähnlich Tricks nötig, um ihn für die Auswertung der Funktion zu ermitteln.

Eine Besonderheit fällt noch bei JDK1.4.2 auf: Dessen XObjects können, wenn man sie einmal nach einem Nodeset gefragt hat, keine anderen Typen mehr zurückgeben. Nachdem dieses Verhalten weder bei JDK1.4.1 noch bei neueren Xalan-Versionen auftritt, gehe ich von einem Bug in JDK1.4.2 aus. Dieser Bug wird im Adapter für JDK1.4.2 dadurch umgangen, dass der Inhalt des XObjects beim ersten Nodeset-Zugriff in ein anderes Objekt kopiert wird. Weitere Anfragen erfolgen dann über dieses Kopie.

Die Adapter für JDK1.4.1 bzw. JDK1.4.2 müssen wegen der undokumentierten API jeweils mit dem JDK compiliert werden, für das sie gedacht sind. Die Standardimplementation der `PXAFactory` wählt diese Adapter zur Laufzeit aus, falls die JDK-Version mit der Version des Adapters übereinstimmt.

Nachdem die dokumentierte XPath-API erst seit Java 1.5 zur Verfügung steht, ist Preference XPath auf älteren Java-Versionen als 1.4.1 nicht lauffähig; sollte aus irgendeinem Grunde ein JDK 1.4.3 veröffentlicht werden, so würde es dort auch nicht laufen. Dies ist aber hinreichend unwahrscheinlich. Unterstützung für 1.4-JDKs, die nicht von Sun sind (beispielsweise von IBM oder Blackdown), kann ich zudem nicht garantieren, da ich nicht weiß, ob diese den Xalan-Parser überhaupt enthalten.

Alle in den folgenden Kapiteln beschriebenen Adapter laufen unter JDK 1.5.0.

4.5.2 Die saubere Lösung

Kommen wir nun zur Kernaufgabe dieser Diplomarbeit – der Implementierung einer Adapterklasse, die mit der offiziellen XPath-API von Java 1.5 oder höher auskommt.

Hierbei traten die meisten Probleme auf. Wie bereits beschrieben, mussten ziemliche Klimmzüge unternommen werden, um die Anforderung dieser API, den Rückgabotyp bereits vor der Auswertung zu kennen, erfüllen zu können.

Erschwerend kam hinzu, dass einige Ergebnisse sogar nacheinander in verschiedenen Rückgabetypen angefragt werden. Die offiziellen Funktionen zum Konvertieren von XPath-Ergebnissen sind für die deshalb nötigen Umwandlungen aber viel zu langsam. Vermutlich verwenden diese intern eine XPath-Anfrage für jede Konversion, was insbesondere wegen der inzwischen bekannten DOM-nach-DTM-Konversion (wenn der Ausgangstyp ein `NodeSet` war) viel Zeit in Anspruch nimmt. Dies führte dazu, dass die Konvertierungen zusammen mehr Zeit brauchten als die eigentliche XPath-Anfrage. Aus diesem Grunde wurde die Konvertierung (nach der Spezifikation in [W3C99]) komplett neu geschrieben.

Wegen der bereits beschriebenen ständigen Neukonvertierung von DOM nach DTM wurde ein *Pruning* implementiert, das bei (Unter-)Anfragen schlichtweg alles außerhalb des Kontextknotens wegschneidet. Allerdings können sich Abfragen auch auf Elemente außerhalb des Kontextknotens beziehen. In unserer fiktiven Schülerdatenbank (die aus Datenschutzgründen in dieser Form nie existieren wird) könnte man, um Schüler für eine ehrenamtliche Aufgabe (die etwas Einarbeitung erfordert) zu finden, eine Anfrage wie

```
/db/schule/klasse/schueler#[(@engagement highest and @ehrlichkeit  
highest) prior to ../@klassenstufe lowest]#/name
```

stellen. Sie findet engagierte ehrliche Schüler, die in einer möglichst niedrigen Klasse sind (damit sich die Einarbeitung möglichst gut rentiert). Wie man an diesem Beispiel schon sieht, lässt sich eine solche Unteranfrage in den meisten (nicht pathologischen) Fällen dadurch erkennen, dass sie mit „../“ oder „../..“ beginnt. Ist einmal für eine andere Unteranfrage ein kompletter Kontext nötig, so kann man diese mit „../“ beginnen lassen – einer Zeichenfolge, die zu „./“ äquivalent ist, aber das Pruning deaktiviert.

Hat man auf die Formulierung der Anfragen keinen Einfluss und ist die Performance nicht wichtig (hier sei schon mal auf Kapitel 5 für Evaluationsergebnisse verwiesen), so kann man durch Laden der `PXAFactoryDANoPruning`-Klasse die Verwendung des Prunings auch komplett deaktivieren.

4.5.3 Steigerung der Performance unter JDK1.5.0

Nachdem die zuletzt beschriebene Implementierung die meiste Zeit (insbesondere bei großen Datenbanken, was eher die Regel als die Ausnahme sein wird) bei der Konvertierung von DOM nach DTM verbraucht, ist eine naheliegende Idee, an dieser Stelle mit der Verbesserung anzusetzen. Dies wird dadurch begünstigt, dass ein DTM-Kontext die Möglichkeit bietet, einen DOM-Knoten in einem bereits konvertierten DTM-Dokument zu suchen (mit der `getDTMHandleFromNode`-Methode).

Diese Optimierung hat den Vorteil, dass sie nur an einer Stelle in die Implementierung eingreift, nämlich beim Aufruf des XPath-Parsers. Wird eine Preference-XPath-Anfrage ausgeführt, so wird die Konvertierung nach DTM selbst übernommen und der resultierende DTM in einem dafür vorgesehenen Feld `optimizationData` des Resolvers gespeichert. Alles weitere – die Konvertierung der Zwischenergebnisse nach DOM, das Aufrufen der Preference-Function, das Auswerten der Präferenzen – läuft wie gehabt. Lediglich die weiteren XPath-Parser-Aufrufe zum Auswerten von Attributselektoren stellen fest, dass bereits ein DTM im Resolver hinterlegt ist. Deshalb generieren sie keinen neuen DTM sondern verwenden die oben genannte Methode, um den momentanen DOM-Kontextknoten im existierenden DTM zu suchen. Die Unteranfragen laufen dann alle auf demselben DTM ab wie die Hauptanfrage. Dass sich dies äußerst positiv auf das Laufzeitverhalten auswirkt (vgl. auch Kapitel 5), ist nicht verwunderlich.

Diese Implementierung hat natürlich wiederum den Nachteil, JDK1.5.0-spezifischen Code zu enthalten. Allerdings besteht dieser Code aus weniger als 100 Zeilen (inkl. Kommentaren und Leerzeilen), sollte also bei Bedarf mit minimalem Aufwand auf neuere Versionen portiert werden können (solange diese immer noch denselben Flaschenhals haben, versteht sich). Und auch wenn diese Abhängigkeit nicht erwünscht

ist, zeigen die Implementation und die damit verbundenen Evaluationsergebnisse eindrucksvoll, dass der wahre Laufzeit-Flaschenhals wirklich hier anzusiedeln ist.

4.5.4 Performanceoptimierte Lösung nur für JDK1.5.0

Andererseits kann es natürlich auch Situationen geben, bei denen ohnehin von vorneherein klar ist, dass sie nur auf JDK1.5.0 laufen werden (Servlet-Lösungen beispielsweise). In diesem Fall kann man erheblich mehr Zeit einsparen als mit gerade einmal 100 Zeilen Java-Code möglich ist. Für diesen Zweck wurde die Implementierung, die in Kapitel 4.5.5 vorgestellt wird, durch Anpassung der Klassennamen automatisiert auf JDK1.5 „portiert“. Offensichtlich stimmt der bei JDK1.5 in der undokumentierten API enthaltene Xalan-Parser im Wesentlichen mit Xalan-J 2.7.0 überein. Die Performance ist auch im wesentlichen identisch mit derjenigen von Java 1.4.x.

4.5.5 Verwendung externer XML-Parser (Apache Xerces/Xalan)

Zu guter Letzt gibt es auch noch die Möglichkeit, einen externen XML-Parser (Xalan-J 2.7.0) zu verwenden. Dies hat den Vorteil, von Java-Versionen weitgehend unabhängig zu sein und trotzdem die gewohnte Performance beizubehalten. Dieser XML-Parser funktioniert unter JDK1.4 und 1.5 und wird voraussichtlich auch unter zukünftigen JDKs funktionieren. In dieser Hinsicht ist diese Lösung also die flexibelste Lösung. Aber auch diese hat einen gravierenden Nachteil: Die Bibliotheken von Xalan brauchen zusammen mit den Xerces-Bibliotheken, auf denen sie aufbauen, etwa 5 MB Plattenplatz. Daher ist die Lösung für Java-Applets oder -Webstart-Anwendungen, sowie für sonstige Programme, die vom Benutzer aus dem Internet heruntergeladen werden müssen, ein deutlicher Nachteil – die Preference-XPath-Bibliotheken inkl. Testframework brauchen nämlich nur etwa 300 KB Plattenplatz. Wenn die Anwendung, die Preference XPath benutzt, also nicht ohnehin schon megabytegroß ist (so dass 5 MB nicht mehr ins Gewicht fallen) ist vermutlich jede andere Lösung vorzuziehen.

4.5.6 Vergleich der Implementationen

Tabelle 4.1 bietet einen Überblick über die verschiedenen Möglichkeiten, die einem bei Verwendung von JDK1.5 offenstehen. Die Performance der einzelnen Lösungen wird im Kapitel 5 untersucht werden.

Implementierung	Kompatibilität	Platzbedarf	Performance
Offizielle API	hoch	normal	schlecht
Minimale Optimierung	mit etwas Aufwand verbunden	normal	gut
Nur für JDK1.5	nicht vorhanden	normal	sehr gut
Externer Parser	hoch	5 MB extra	sehr gut

Tabelle 4.1: Überblick über die verschiedenen Implementationen

4.6 Buildfile

Nachdem der Compiliervorgang durch die vielen Adapter und ihre Abhängigkeiten zu gegebenermaßen unübersichtlicher geworden ist, wurde unter Verwendung von *Apache Ant* ein Buildfile erzeugt, das den kompletten Buildprozess, angefangen vom Erzeugen des Parsers mit JavaCC, bis zum Verpacken der Klassen in `jar`-Dateien, übernimmt. Dies macht einerseits das Erzeugen von neuen Releases einfacher, andererseits werden typische Fehler wie ein vergessener JavaCC-Aufruf dadurch verhindert (wenn jeder das Buildfile benutzt). Nachdem Ant (ähnlich Make) über eine Abhängigkeitsprüfung verfügt, wird hierbei auch nur das neu gebaut, was sich wirklich geändert hat. Dieser Buildvorgang integriert sich nahtlos in die meisten Java-Entwicklungsumgebungen, ist andererseits aber auch von der Entwicklungsumgebung unabhängig.

Ein Nachteil des automatischen Builds besteht darin, dass man zum Bauen neben Ant auch noch alle anderen nötigen Build-Utilities (also in diesem Fall JavaCC sowie drei verschiedene Java-SDKs) benötigt. Allerdings kann man die entsprechenden Teilvorgänge auch aus dem Buildvorgang ausnehmen, wenn man beispielsweise keine JDK1.4-Unterstützung braucht oder den JavaCC-Parser nicht verändert hat.

Will man die Adapter aus irgendeinem Grunde per Hand bauen, so muss man berücksichtigen, dass die Minimalimplementierung (aus Kapitel 4.5.3) auf der offiziellen Implementierung (aus Kapitel 4.5.2) aufbaut. Weiterhin verwenden die JDK-1.4.x-Implementierungen (Kapitel 4.5.1) und die Xalan-Implementierung (Kapitel 4.5.5) Code aus dem `preference.xpath.adapter.xalanbase`-Paket, der also vorher gebaut werden oder im Classpath vorliegen muss. Sämtliche Adapter hängen des Weiteren natürlich sowohl von den Basisklassen als auch von den JDK-unabhängigen Preference-XPath-Klassen ab.

5 Performance-Analysen

5.1 Verwendete Datenbanken

Zum Testen von Initialisierungsoverheads wurde eine Datenbank namens SMALL verwendet, die ihrem Namen alle Ehre macht. Ihr kompletter Inhalt sieht nämlich so aus:

```
<?xml version="1.0" encoding="UTF-8"?>
<db><elem value="42"/><elem value="1"/></db>
```

Des Weiteren existierte bereits für die Testfälle eine Datenbank AUTOS, die die technischen Daten von 20 (vermutlich fiktiven) Automobilen enthält und insgesamt 8 KB groß ist. Eine weitere bereits existierende Datenbank COSIMA enthält neben einigen Metadaten Informationen über 860 verschiedene Kästen und ähnliche „Lager- und Betriebseinrichtungen“ in 28 Kategorien und hat einen Umfang von 1,6 MB.

Für einige Tests ist es sinnvoll, ähnlich strukturierte Datenbanken in verschiedenen Größen vorliegen zu haben, wobei sowohl die Datensatzgröße als auch die Datensatzanzahl variabel sein sollte. Für diesen Zweck wurden dynamisch Pseudodatenbanken $\text{DUMMY}_k^\otimes(n)$ (für $\otimes \in \{+, 0, -\}$) angelegt. Diese besitzen jeweils n Datensätze von gleicher Struktur. Jeder Datensatz besitzt zunächst mal eine numerische ID. Diese IDs sind für $\otimes \equiv +$ aufsteigend (1 bis n), für $\otimes \equiv -$ absteigend (n bis 1) bzw. für $\otimes \equiv 0$ konstant (0) nummeriert. Jede ID taucht sowohl als Attribut im Wurzelement des Datensatzes auf, als auch als Attribut eines Kindelements; bei Zugriffen auf das Kindelement wird nämlich zusätzlich Zeit benötigt, um das richtige Kindelement im Datensatz zu finden. Des Weiteren besitzt jeder Datensatz noch k identische Pseudoelemente, die verwendet werden, um Abhängigkeiten von der Datensatzgröße zu analysieren.

Die Datenbank $\text{DUMMY}_2^+(3)$ würde also folgendermaßen aussehen:

```
<?xml version="1.0" encoding="UTF-8"?>
<db>
  <record value="1">
    <name value="1"/>
    <lorem ipsum="dolor sit amet"/>
    <lorem ipsum="dolor sit amet"/>
  </record>
  <record value="2">
    <name value="2"/>
    <lorem ipsum="dolor sit amet"/>
    <lorem ipsum="dolor sit amet"/>
  </record>
  <record value="3">
    <name value="3"/>
    <lorem ipsum="dolor sit amet"/>
    <lorem ipsum="dolor sit amet"/>
  </record>
</db>
```

Für die Tests wurden, um repräsentative Ergebnisse zu erhalten, natürlich wesentlich größere Datenbanken verwendet (tausende Datensätze mit je hundert von Pseudoelementen).

5.2 Testumgebung

Als Testumgebung diente ein Pentium IV mit 3 GHz Taktfrequenz und 1 GB RAM. Die einzelnen Tests liegen als Java-Programm vor, das gegen die verschiedenen Adapterimplementationen „gelinkt“ wird und dann jeden Test fünfmal ausführt, wobei vor jeder Ausführung der Garbage-Collector und der Finalizer manuell angestoßen wird¹. Die einzelnen Werte für denselben Test lagen meist nur wenige Millisekunden auseinander. Für die Diagramme im folgenden Abschnitt wurde jeweils der Median der 5 Werte (also derjenige Wert, der bei aufsteigender Sortierung „in der Mitte steht“) benutzt, weil dieser eventuell auftretende starke Abweichungen eines einzelnen Messwerts, die auf heutigen modernen Systemen nie gänzlich auszuschließen sind, weniger gewichtet als der Mittelwert.

¹Man kann in der Theorie keine Garbage Collection erzwingen. In der Praxis führt eine Sekunde Inaktivität nach dem Empfehlen der Garbage-Collection aber zumindest bei Suns Java dazu, dass der Garbage-Collector auch wirklich durchläuft

Bezeichnung	Bedeutung
original	Version von Preference XPath, die vor meiner Diplomarbeit vorlag
150DTM	Portierung der Version für Java 1.4.x, vgl. Kapitel 4.5.4 (verwendet DTM)
ApacheDTM	Verwendung eines externen Apache Xalan, vgl. Kapitel 4.5.5 (verwendet DTM)
ApacheDOM	Verwendung eines externen Apache Xalan, vgl. Kapitel 4.5.5 (wobei DOM verwendet wird)
150Minimal	Implementierung mit minimalem 1.5.0-spezifischem Code, vgl. Kapitel 4.5.3
DA	Verwendet nur dokumentierte API, vgl. Kapitel 4.5.2, aber beschleunigt durch Pruning
NoPruning	Verwendet nur dokumentierte API, vgl. Kapitel 4.5.2, und kein Pruning

Tabelle 5.1: Bezeichnungen für Adapterimplementationen in Diagrammen

In den folgenden Diagrammen werden die einzelnen Implementierungen wie in Tabelle 5.1 aufgeführt abgekürzt.

Da es bei den Tests weniger um die Basisklassen als um die XML-Parser-Anbindung geht, sind die Anfragen bewusst einfach gehalten (HIGHEST-, POS- oder NEG-Anfragen). Die genauen Anfragen und alle Testergebnisse können Anhang C entnommen werden.

5.3 Lastverteilung

Nachdem die Preference-XPath-Implementierungen allesamt im Speicher arbeiten (die XML-Datenbank wird vor Ausführung einer Anfrage komplett in den RAM geladen), sind Anfragen extrem CPU-lastig. Nachdem das Testsystem inklusive Datenbank komplett im RAM des Testsystems Platz hatte, traten während der Tests auch kaum Festplattenzugriffe auf (außer zum Schreiben des Testprotokolls). Die CPU-Last befand sich während der Tests konstant bei 100%.

Bei der stichprobenweisen Analyse der Tests mit einem Profiler stellte sich heraus, dass der größte Anteil der Rechenzeit (über 75%, meistens aber deutlich mehr) vom XML-Parser verbraucht wird. Die in den Algebraklassen notwendigen Vergleiche sind also offensichtlich erheblich performanter als das Parsen des XML-Dokuments.

5.4 Testergebnisse

5.4.1 Initialisierungszeiten

Bei Serveranwendungen ist sie zu vernachlässigen, bei Desktopanwendungen hingegen umso wichtiger: Die Ladezeit, die nötig ist, um das Programm zu initialisieren. Braucht ein Applet auf einer Webseite mehr als ein paar Sekunden zum Laden, sucht sich der Besucher schnell eine andere Seite aus. Initialisierungszeiten wurden nach dem Start der Laufzeitumgebung gemessen; die Ladezeit der Laufzeitumgebung selbst ist unabhängig vom Programm, also auch von der Preference-XPath-Implementierung.

In dieser Hinsicht sind die Unterschiede zwischen den Implementierungen aber nur marginal: Die Initialisierung des internen Parsers läuft quasi in Nullzeit ab (unter 100 Millisekunden), die Initialisierung der externen Apache-Implementierung dauert um die 300 Millisekunden, die vermutlich hauptsächlich für die Bytecodeverifikation der Apacheklassen verbraucht werden.

Abbildung 5.1 zeigt die Initialisierungszeiten graphisch. Hierbei ist der untere Teil des Balkens die Zeit, die für die Initialisierung einer nicht präferenzbasierten Abfrage benötigt wird; der obere Teil zeigt die Zeit, die eine präferenzbasierte Abfrage zusätzlich benötigt.

5.4.2 Anfragen an kleine Datenbanken

Alle Anfragen in diesem Abschnitt wurden mehrfach in Reihe ausgeführt, da die Zeiten ansonsten nicht messbar waren. Dabei wurde die Häufigkeit so gewählt, dass die Anfragen zusammen etwa zwischen 500 und 1500 Millisekunden brauchten – kurz genug, um den Test viele Male auszuführen, aber auch lang genug, um Unterschiede messen zu können. Wir wollen im Folgenden nur noch die Implementationen betrachten, die unter JDK1.5 zur Verfügung stehen (die anderen stellen keine direkte Alternative da). Als Vergleichswert wird jeweils noch die Zeit unter Verwendung der „Originalimplementierung“ (also der Implementierung, die vor meiner Diplomarbeit vorlag) angegeben.

Der erste Test, eine Anfrage auf der Autodatenbank nach einem Auto mit maximalem Verbrauch, wurde jeweils 250 mal ausgeführt. Für die Darstellung der Messwerte in Abbildung 5.2 wurde der Median dann wieder durch 250 dividiert, um die durchschnittliche Zeit einer Anfrage zu erhalten. Nachdem durch die starke Streuung der Testergebnisse (die bei den folgenden Tests noch auffälliger wird) die kleineren Ergebnisse kaum unterscheidbar sind – eine logarithmische Skala wäre allerdings in diesem

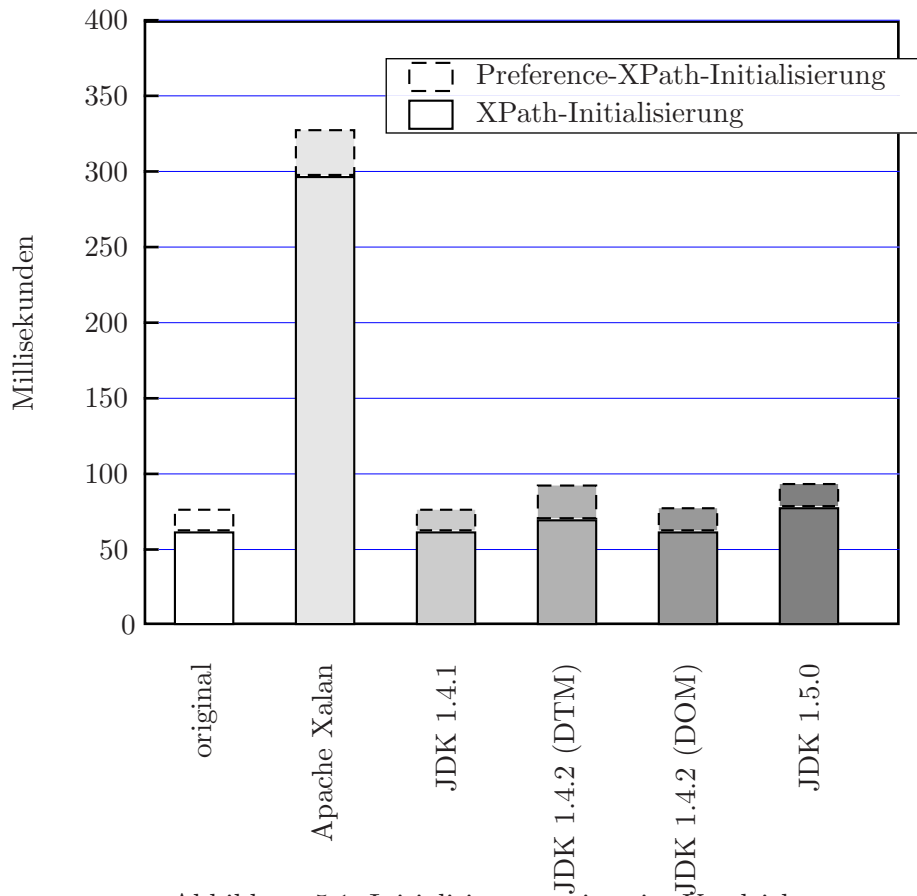


Abbildung 5.1: Initialisierungszeiten im Vergleich

Kontext irreführend –, enthält diese Abbildung eine weitere Grafik, die die Werte „gezoomt“ darstellt.

Hier sieht man recht schön, dass Pruning kein Allheilmittel darstellt; ob mit oder ohne Pruning, die Anfrage dauert bei Verwendung nur dokumentierter APIs jedesmal um die 30 Millisekunden. Alle Implementierungen, die die zusätzliche Konvertierung von DOM nach DTM gänzlich umgehen, kommen mit weniger als 5 Millisekunden aus. Dieser Effekt besteht bei größeren Datenbanken sicher auch, nur fällt er bei längeren Abfragezeiten nicht mehr so stark ins Gewicht. Andererseits kommt bei derart kleinen Anfragen auch der „konstante Term“ der Laufzeiten mehr zum Tragen: Implementierungen, die „zu Tode optimiert sind“ (wie die Apache-Implementationen oder die von Java 1.4 portierte Lösung) schneiden hier schlechter ab als die ursprüngliche Implementierung sowie die recht einfache „minimale Implementierung“ (die versucht, mit minimalem Codezeilenaufwand maximale Geschwindigkeit zu erreichen).

Dieses Ergebnis sollte man aber auch nicht überbewerten; 30 Millisekunden sind eine so kurze Zeit, dass sie einem Benutzer nie auffallen wird.

Führt man auf unserer Minimaldatenbank mit 2 Datensätzen Anfragen aus, so bewegen sich die Anfragezeiten zwischen 2 und 3 Millisekunden. Keine der Implementierungen enthält also einen nennenswerten Overhead, der bei Verwendung dieser Implementierung immer auftreten würde. Auffällig in diesem Fall ist, dass von den von mir durchgeführten Tests die Anfrage ohne Präferenzen (eine reine harte Anfrage) die einzige ist, bei der die Originalimplementation anscheinend am besten abschneidet. Hierbei sollte man aber auch berücksichtigen, dass die Originalimplementation unter Java 1.4 ausgeführt wird, alle anderen Beispiele dieser Diagramme aber unter Java 1.5 liefen. Führt man dieselbe Anfrage mit der neuen Java-1.4.1-Implementierung aus, so ist auch hier (wie man dem Anhang entnehmen kann) die neue Implementierung schneller – der Overhead, um Daten in ThreadLocals zu speichern, ist schließlich unabhängig davon, ob die Daten dort jemals benutzt werden. Warum diese kleine Anfrage unter Java 1.4 durch die Bank schneller ist als unter Java 1.5, ließ sich nicht klären.

5.4.3 Anfragen an große Datenbanken

Präferenzanfragen bekommen gerade bei größeren Datenbeständen ihren Sinn – kleinere Datenbestände (unter 100 Datensätze) kann man in Ermangelung einer sinnvollen Suchfunktion noch per Hand durchgehen, bei größeren Datenbeständen führt dies schnell zur Frustration. Deshalb ist es wichtig, dass Preference-XPath-Anfragen auch auf großen Datenbanken effizient ausgeführt werden können. Beispielhaft werden hier drei Anfragen genauer untersucht: Zwei Anfragen auf der COSIMA-Datenbank (860 Datensätze) und eine Anfrage auf einer DUMMY-Datenbank mit 1000 Datensätzen.

1. Die Anfrage COSIMA-A ist ein recht extremes Beispiel, da sie alle Datensätze bis auf einen als Ergebnistreffer aufweist. Selbst in diesem extremen Fall, bei dem die Präferenzalgebraklassen am meisten zu tun haben (die Anzahl der XPath-Anfragen ist durch das Caching konstant), ist der Unterschied zwischen den Implementationen mehr als deutlich. Die schnellen Implementationen (welche die undokumentierte API oder eine externe Xalan-Bibliothek verwenden) brauchen für eine Anfrage weniger als eine halbe Sekunde, die unoptimierte JDK1.5-Implementierung, die nur dokumentierte Funktionen verwendet, braucht hingegen knapp 45 Sekunden (das ist die 90-fache Zeit). Pruning (was die Semantik verändert) oder die Verwendung eines Präferenzkontexts (was undokumentierten Code benötigt) drückt diese Zeit auf etwa 2,5 Sekunden, was zwar immer noch mehr ist als die „bestmöglichen“ 500 Millisekunden, aber durchaus noch akzeptabel. Die Messergebnisse sind in Abbildung 5.3 zu sehen.

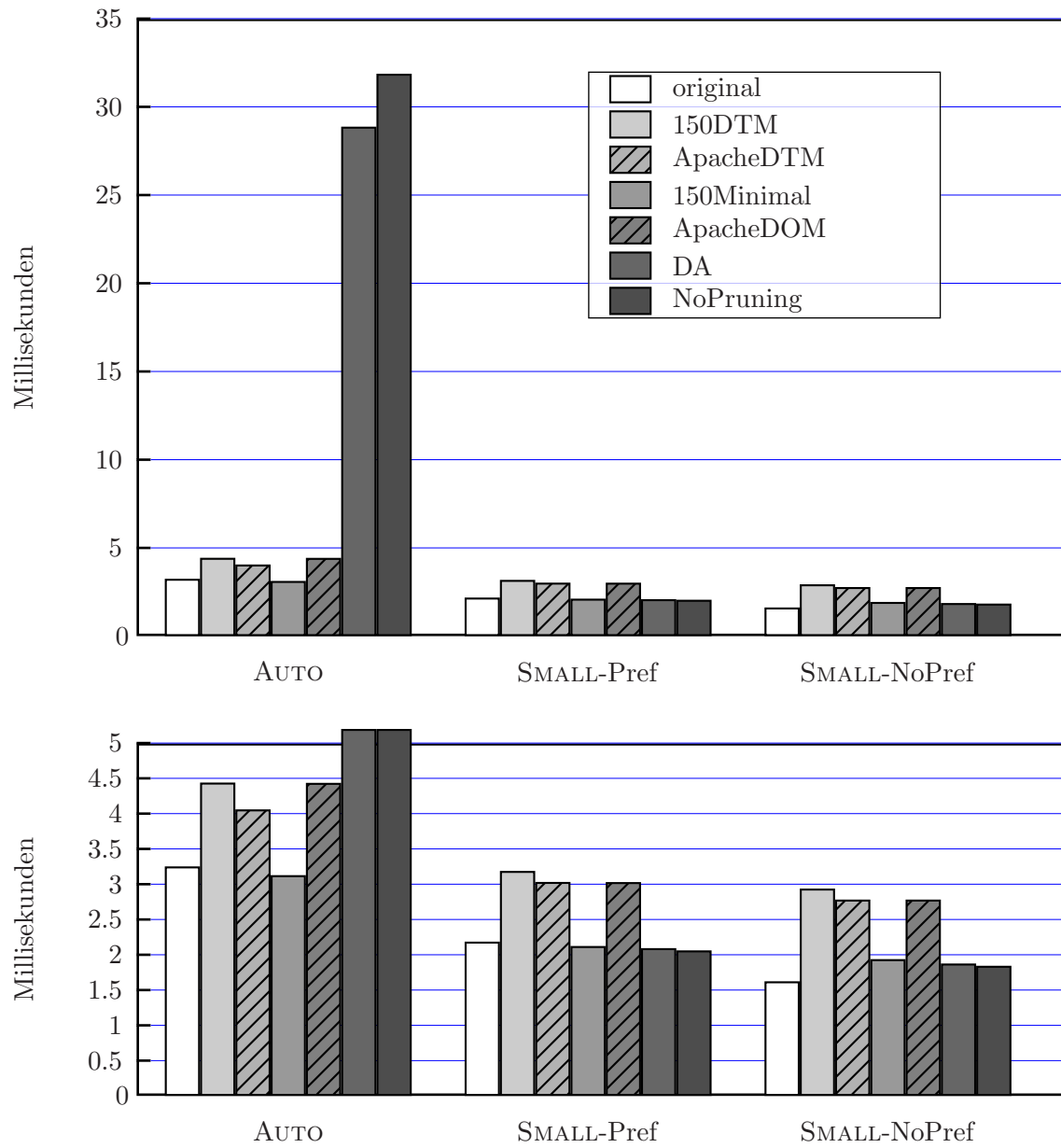


Abbildung 5.2: Auswertungszeiten einiger Anfragen auf kleinen Datenbanken (unten: abgeschnittene vergrößerte Darstellung)

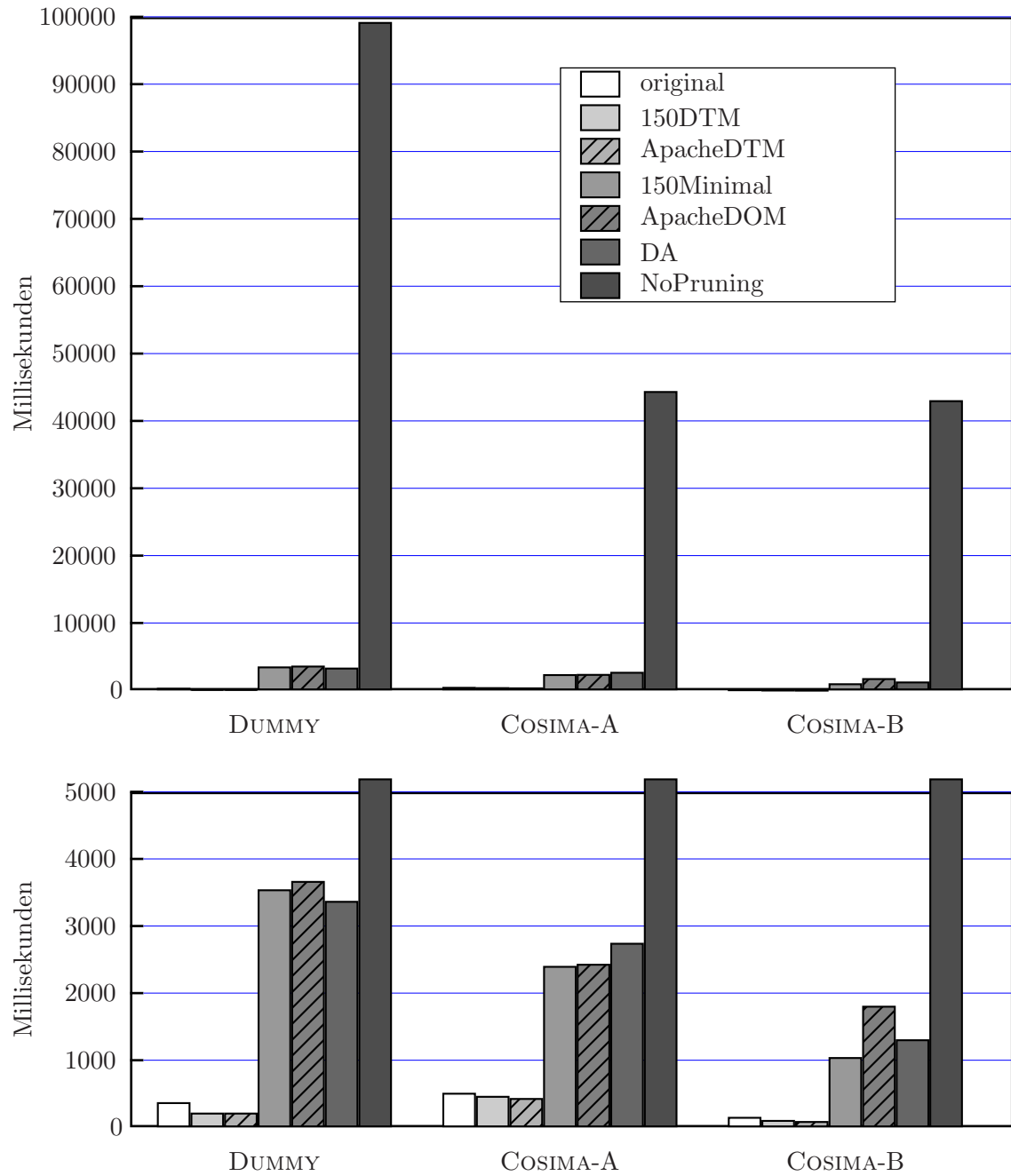


Abbildung 5.3: Auswertungszeiten einiger Anfragen auf großen Datenbanken. (unten: abgeschnittene vergrößerte Darstellung)

2. Eine andere Anfrage aus der COSIMA-Datenbank (COSIMA-B), die in einer weichen Anfrage per ID den ersten Datensatz selektiert (zugegeben, eine derartige Anfrage würde man in der Praxis als harte Anfrage formulieren), stellt das andere Extrem dar: Hier beschränkt sich die Arbeit die Präferenzalgebraklassen auf das Behalten des ersten Datensatzes. Entsprechend werden die Abfragezeiten kürzer: Nun reicht die Spanne von 110 Millisekunden (undokumentierte API) bis zu 43 Sekunden (dokumentierte API).
3. Betrachten wir nun die Anfrage `/db/record#[name/@value highest]#` auf der DUMMY₁₀₀⁺(1000)-Datenbank. Auch hier ist die Anfrage bei den schnellsten Implementierungen nach 200 Millisekunden ausgewertet. Die reine JDK1.5-Implementierung braucht diesmal sogar knapp 100 Sekunden (mehr als eineinhalb Minuten!). Pruning oder die Wiederverwendung des DOM-Kontexts bewirken eine Laufzeit von etwa 3,5 Sekunden. Gerade dieses Beispiel zeigt die Laufzeitunterschiede zwischen den Implementierungen am deutlichsten – niemand will wirklich für eine Abfrage auf 1000 Datensätzen eineinhalb Minuten warten, wenn er die Antwort auch in 200 Millisekunden haben kann.

Das Diagramm enthält auch Messwerte für die Verwendung der externen Apache-Bibliothek, bei denen im präferenzspezifischen Teil keinerlei DTM-Code verwendet wird, der ansonsten aber mit der normalen Implementierung für den externen Adapter identisch ist. Nachdem die Zeiten für diese Implementierung in etwa mit denen im Mittelfeld übereinstimmen, sind Adapter, die intern nicht mit DTM arbeiten, wohl in der Praxis nicht schneller zu bekommen.

Die eigentlichen Anfragen sind also jeweils im Millisekundenbereich beantwortbar; die Verwendung von DOM statt DTM kostet ein paar weitere Sekunden. Erzwingt man jedoch für jede Unterabfrage eine komplette DOM-nach-DTM-Konvertierung, so explodiert der Zeitaufwand in den Minutenbereich.

Bei der Betrachtung von Abbildung 5.3 fällt ferner auf, dass der Zeitunterschied zwischen COSIMA-A und COSIMA-B bei den einzelnen Implementierungen durchaus verschieden ist, obwohl der einzige Code, der sich hierbei anders verhält (der Code in den Präferenzalgebraklassen), identisch ist. Die einzige mir plausibel erscheinende Erklärung für diesen Umstand ist, dass sich durch die größeren BMO-Sets der freie Speicher soweit verringert, dass häufigere Garbage-Collections während des XML-Parsens nötig werden.

5.4.4 Abhängigkeit der Laufzeit von der Datenbankgröße

Für diesen Test wurden Dummy-Datenbanken in verschiedenen Größen generiert und Anfragen auf diesen ausgeführt. Da wir hier Veränderungen vergleichen wollen und keine absoluten Laufzeiten, wurde eine Anfrage auf einer $\text{DUMMY}_{100}^0(500)$ als Basis für die anderen Anfragen verwendet – diese Anfrage stellt also immer den Wert 100% dar.

Ändert man die Menge der Dummydaten im Datensatz, so verlängert sich die Laufzeit in etwa linear. Bei Verwendung von DTM wird sie gar mehr als verdoppelt, bei Pruning etwas weniger. Die genaue Ursache hierfür ist mir unbekannt, allerdings zeigt es recht eindrucksvoll, dass es wichtig ist, möglichst wenig irrelevante Daten in der Datenbank (innerhalb der Datensätze) zu haben. Diese sollten besser vorher (beispielsweise per XSLT) aus der Datenbank entfernt werden. Abfragen auf Attribute der Wurzel des Datensatzes (also Abfragen der Form `/db/pfad#[@attr highest]#`) sind schneller als Abfragen auf Attribute von Unterelementen (wie `/db/pfad#[elem/@attr highest]#`). Dies kann man durch eine geeignete Datenbankstruktur (wichtige Werte in Attribute der Wurzel speichern) unterstützen.

Verdoppelt man die Anzahl der Datensätze, so führt das auch mindestens zu einer Verdoppelung der Laufzeit. Verwendet man bei der dokumentierten JDK1.5-Implementierung kein Pruning, so vervierfacht sich die Laufzeit in etwa, da ein doppelt so langes XML-Dokument doppelt so oft von DOM nach DTM konvertiert werden muss.

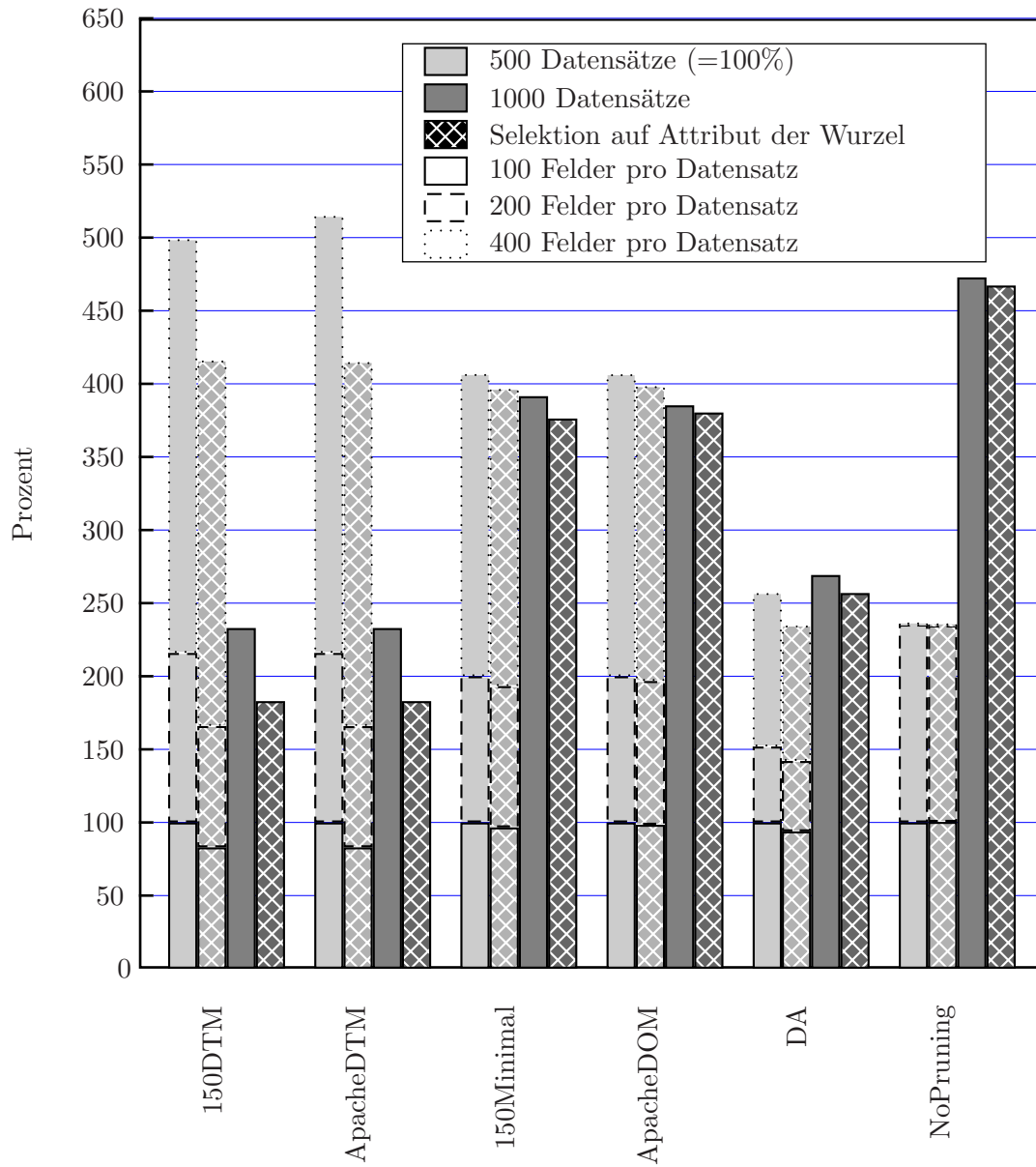


Abbildung 5.4: Abhängigkeit der Laufzeit von der Datenbankgröße

6 Zusammenfassung und Ausblick

6.1 Zusammenfassung

Die Anpassung von Preference XPath an JDK 1.5.0 lief bei weitem nicht so geradlinig, wie man es hätte erwarten können. Eine Implementation, die lediglich dokumentierte API verwendet, ist zwar möglich, aufgrund ihres schlechten Laufzeitverhaltens aber praktisch kaum zu verwenden.

Eine mögliche Lösung ist die Verwendung eines externen XPath-Parsers; an diesen Parser angepasster Code zeigt ein akzeptables Laufzeitverhalten, ist aber wegen der großen Bibliotheken oft inakzeptabel. Andere Lösungen bestehen darin, den Code an wenigen Stellen doch undokumentierte API verwenden zu lassen, was ein ähnliches Laufzeitverhalten zeigt, aber bei neuen Versionen möglicherweise Anpassungsarbeiten erfordert. Eine dritte Möglichkeit wäre, unter neueren JDK-Versionen transparent auf die langsame Implementierung zurückzufallen, bis einschließlich 1.5.0 aber die optimierte Lösung zu verwenden. Je nach Anwendungsfall sind verschiedene Lösungen zu bevorzugen.

Die Anpassungen der Basisklassen machen die Verwendung von Layered-Präferenzen und SV-Relationen übersichtlicher bzw. im Fall von Preference SQL erst möglich. Dies erlaubt dem Benutzer, Präferenzen feinkörniger auszudrücken als bisher.

Bei der Anpassung zahlte es sich aus, softwaretechnische Methoden (Patterns) zu benutzen. Die Kapselung der Implementierung erlaubt es, viele verschiedene Implementierungen auszuprobieren und gleichzeitig implementationsunabhängige Programmteile (und mögliche Fehler darin) nicht zu duplizieren. Dynamisches Austauschen der Implementierung zur Laufzeit macht es möglich, eine Anwendung bereitzustellen, die trotz unterschiedlichster API unter Java 1.4 und Java 1.5 gleichzeitig laufen kann. Weitere Patterns (wie das *Templatemethoden*-Pattern) helfen dabei, duplizierten Code zu vermeiden.

Die Verwendung von Versionskontrollsystemen wie CVS ist, wenn mehrere Leute am selben Code arbeiten, fast schon ein Muss. Wenn jemand keine Versionskontrolle verwendet, sondern stattdessen munter an alten Versionen weiterarbeitet, kann er durchaus einiges an Arbeit (für das Einbinden seiner Änderungen) verursachen.

Zu guter Letzt zeigte das Profiling wieder mal, dass textbasierte plattformunabhängige Technologien wie XML, so praktisch sie für den Datenaustausch sein mögen, für interne Auswertungen besser gemieden werden sollten. Denn das Parsen von XML macht, trotz der Existenz von Modellen wie DTM, immer noch den überwiegenden Teil der Zeit einer XML-Anfrage aus.

6.2 Ausblick

Das Java-SDK ist bei 1.5 nicht an einem Ende angelangt. Gerade wird fieberhaft am Nachfolger (Codename *Mustang*) gearbeitet, der nach momentanem Entwicklungsstand im Herbst 2006 veröffentlicht werden soll.

Was die Java-Zukunft bringen wird, weiß niemand genau. Es ist aber durchaus möglich, dass die Implementierung der XPath-API, die ja in Java 1.5 ganz neu ist, noch performanter wird – in diesem Fall wäre dann die langsame Implementierung möglicherweise doch gar nicht mehr so langsam.

Eine andere Optimierungsmöglichkeit bestünde darin, ein internes Datenformat zu entwickeln, das nicht unbedingt XML-basiert sein muss, dafür aber auf die Abfrage von Präferenz-Anfragen – viele XPath-Unteranfragen (mit skalarem Ergebnistyp) auf Kindelementen eines bestimmten Knotens – optimiert ist. Eine Form von „Indizes“ (die für eine bestimmte Anfrage auf allen Kindelementen eines Knotens die Ergebnisse sortiert speichert) könnte hierbei die Anfrage vermutlich wesentlich beschleunigen.

Literaturverzeichnis

- [ASF05a] The Apache Software Foundation. *Apache Xindice*, 2005. <http://xml.apache.org/xindice/>, abgerufen am 20.04.2006.
- [ASF05b] The Apache Software Foundation. *Xalan-Java DTM*, 2005. <http://xml.apache.org/xalan-j/dtm.html>, abgerufen am 20.04.2006.
- [GHJV94] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. *Design Patterns – Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [Haf01] Bernd Hafenrichter. Preference XPATH: Ordering of set valued Attributes. 2001.
- [KFD04] Werner Kießling, Stefan Fischer, and Sven Döring. COSIMA B2B - sales automation for E-procurement. In *CEC*, pages 59–68. IEEE Computer Society, 2004.
- [Kie02] Werner Kießling. Foundations of preferences in database systems. In *VLDB*, pages 311–322, 2002.
- [Kie05] Werner Kießling. Preference Queries with SV-Semantics. In Jayant R. Haritsa and T. M. Vijayaraman, editors, *COMAD*, pages 15–26. Computer Society of India, 2005.
- [LDI06] Lehrstuhl für Datenbanken und Informationssysteme. *Preference XPath – Tutorial*, 2006.
- [W3C98] The World Wide Web Consortium. *Document Object Model (DOM) Level 1 Specification*, 1998. <http://www.w3.org/TR/REC-DOM-Level-1/>.
- [W3C99] The World Wide Web Consortium. *XML Path Language (XPath), Version 1.0*, 1999. <http://www.w3.org/TR/xpath>.

Anhang A

Inhalt der CD

Auf der beiliegenden CD befinden sich Quellcode, Binaries und API-Dokumentation von Preference XPath (inkl. GUI und Testfällen). Des Weiteren liegen dort die Testprotokolle der Effizienztests, eine PDF-Version dieser Diplomarbeit und Windows-Versionen aller Programme, die für ein erfolgreiches Compilieren von Preference XPath erforderlich sind.

Die Verzeichnisstruktur sieht im einzelnen folgendermaßen aus:

Pfad	Inhalt
/source	Hier liegen die Quelltexte von Preference XPath. Die einzelnen Programmteile liegen in Unterverzeichnissen. Die Datei <code>build.xml</code> in diesem Verzeichnis ist das Haupt-Buildfile. Für einen erfolgreichen Build sind ggf. noch die Pfade in <code>/source/PreferenceXPath/ant.properties</code> anzupassen.
/release	Hier befinden sich die <code>.jar</code> -Dateien der Distribution. <code>pxpath.jar</code> und <code>pxpath.bat</code> reichen für die Ausführung der Beispiel-GUI unter Java 1.4.1, 1.4.2 und 1.5.0 aus. Die weiteren Jar-Dateien enthalten weitere Adapterimplementationen, Unit-Tests bzw. die Xindice-Anbindung.
/javadoc	Hier liegt die generierte API-Dokumentation von Preference XPath. Sowohl <code>/release</code> als auch <code>/javadoc</code> können vom Buildfile in <code>/source</code> generiert werden.
/evaluation	In diesem Verzeichnis liegen sowohl die Programme zur Ausführung und Auswertung der Testfälle als auch die dabei entstandenen Daten – sowohl als Rohdaten, als auch als <code>.csv</code> -Dateien zur Weiterbearbeitung in Tabellenkalkulationen. Im Verzeichnis <code>generiert</code> liegen generierte Zwischendateien für die Analyse der Initialisierungszeiten (vgl. Kapitel 5.4.1).
/pdf	Hier liegt die PDF-Version dieser Diplomarbeit.
/lib	Um das Compilieren zu vereinfachen, liegen hier die nötigen Bibliotheken (Xalan, Xindice, JUnit) in ausgepackter Form.
/tools	Hier liegen Windows-Versionen der benötigten Tools (JDK 1.4.1, JDK 1.4.2, JDK 1.5.0, Xindice, OpenORB, JavaCC, Xalan, Ant, JUnit). All diese Programme können auch kostenlos aus dem Internet heruntergeladen werden.

Anhang B

Weitere Änderungen am Programmcode

Dieser Anhang listet kleinere Programmänderungen auf, die mit dem eigentlichen Diplomarbeitsthema nichts zu tun hatten. Nachdem diese Vorteile bei der Weiterentwicklung bzw. Usability bieten, wurden sie dennoch durchgeführt.

B.1 Kleinigkeiten

Die etwas missverständlich benannte Klasse `DOMExpr` wurde nach `DOMAttributeSelector` umbenannt.

`XPathStringBuilder` konnte eine geparste XPath-Expression wieder in den ursprünglichen String umwandeln. Nachdem diese Funktionalität nicht benötigt wird (man kann die Präferenzen schließlich auch nach XML serialisieren) und die Klasse lange nicht mehr aktualisiert wurde (und deshalb einige der neueren Präferenzen noch gar nicht kannte) wurde sie entfernt.

Der `MultiDocumentMode` konnte bisher – da `Xindice` präferenzlose Anfragen intern ausführen konnte – keine Anfragen mit Präferenzen bearbeiten. Dieses Manko wurde behoben.

Die `ScoreD`-Extension (die eine `SCORE`-Präferenz mit *d*-Parameter implementierte) hatte ihre `Score`-Funktion falsch geladen.

Bisher hat die `PREFERENCE`-Funktion vergessen, die verwendeten Iteratoren (die nicht immer am Anfang waren) zu resetteten. Dies führte dazu, dass manchmal ein paar Ergebnisse übersprungen wurden.

B.2 Anpassungen des Parsers

Zeitgleich zu meiner Diplomarbeit hat ein anderer Diplomand den Parser von Preference XPath (und Preference SQL) um die neuen Funktionen der Basisklassen (SV-Relationen, *d*-Parameter, LAYERED-Präferenz) erweitert.

Weitere Parserfehler wurden von mir im Parser gefunden und behoben: Eine Kleinigkeit war beispielsweise, dass bei der Verwendung von benutzerdefinierten XPath-Funktionen der Doppelpunkt zwischen Präfix und Funktionsname beim Rewriting verloren ging. Des Weiteren führten geschachtelte Anfragen (eine Präferenzanfrage innerhalb einer harten Anfrage oder umgekehrt) zu Abstürzen des Parsers; nachdem derartige Anfragen in der Praxis durchaus vorkommen können, wurde dies ebenfalls behoben. Derselbe Fehler trat bei Präferenzabfragen innerhalb von Funktionsaufrufen (an einer Stelle, an der ein NodeSet erwartet wird) auf.

Ein weiteres Problem besteht darin, dass die Namen von Elementknoten nicht mit Preference XPath-Schlüsselwörtern übereinstimmen durften. Nachdem immer wieder neue Schlüsselwörter hinzukommen (im Rahmen der Anpassungen beispielsweise **regular**, **sv**, **layered**, **others**) und man eventuell Anfragen auf Dokumenten machen will, die nicht extra für Preference XPath erzeugt wurden, stellt dies eine große Einschränkung dar. Dieses Problem ließ sich, da die meisten Schlüsselwörter nur an bestimmten Stellen im Code auftreten können, insbesondere nach einem Slash nie ein Schlüsselwort steht, recht einfach lösen. Einen Spezialfall stellt das **external**-Schlüsselwort dar, da es direkt am Anfang einer Anfrage stehen kann und nicht durch Klammern oder ähnliches klar macht, dass es ein Schlüsselwort ist. Folgende Anfrage ist sogar zweideutig:

```
/db/feld#[external in('text')]
```

Einerseits kann es bedeuten, dass die externe Funktion **in** (die im Repository serialisiert ist) auf dem String **'text'** ausgeführt werden soll, andererseits könnte es sich auf das Feld **external** beziehen, das bevorzugt gleich **'text'** sein sollte. Dieser Fall wird in der ersten Weise interpretiert, um die zweite Interpretation zu erhalten, muss man folgende Form verwenden:

```
/db/feld#[./external in('text')]
```

Bei Verwendung des Vereinigungsoperators **|** unter Xindice wurde einer der beiden zu vereinigenden Teile falsch geparst.

B.3 Testframework, Beispiel-GUI

Für Preference XPath existiert eine Test-GUI, die es ermöglicht, Preference XPath-Abfragen auf XML-Dateien auszuführen. Zusätzlich zum Ergebnis (als Plaintext) werden von dieser auch noch Timinginformationen angezeigt. Weiterhin ermöglicht es die Test-GUI, die umgeschriebenen reinen XPath-Anfragen zusammen mit dem Preference-Array als XML auszugeben, was erheblich bei der Fehlersuche hilft. Queries können abgespeichert werden, um beim nächsten Start wieder verfügbar zu sein. Fehler im Preference-Framework wurden mit „nichtssagenden“ Dialogfeldern angekündigt, die Grundkonfiguration (Pfade, Datenbank, Queries) wurden beim Start aus einer XML-Datei eingelesen.

Jetzt ist das Programm beim Start fehlertolerant – es startet auch komplett ohne Konfigurationsdatei. Änderungen an der Konfiguration werden beim Beenden in die Konfigurationsdatei gesichert. Der Inhalt der Liste der favorisierten Anfragen lässt sich nun komfortabel bearbeiten – einzelne Anfragen lassen sich verschieben oder löschen. Laden und Speichern der Favoritenliste geht weiterhin; zudem wird der Inhalt der Favoritenliste nun beim Beenden in die Konfigurationsdatei gespeichert (und nicht in die Datei, aus der sie zuletzt geladen wurde). Zudem kann eingestellt werden, ob neue Anfragen automatisch in dieser Liste landen sollen. Queries können nun auch mit der Eingabetaste abgeschickt werden, nicht nur per Mausklick. Auftretende Exceptions erscheinen in einem eigenen Dialogfenster inklusive komplettem Stacktrace, was die Fehlersuche vereinfacht.

Neben der Auswertung von einzelnen Abfragen erlaubt die Test-GUI auch noch, so genannte Serienauswertungen zu starten. Eine Serienauswertung ist eine XML-ähnliche Datei, die mehrere XPath-Anfragen zusammen mit den resultierenden XML-Fragmenten enthält. Als Ergebnis einer Serienauswertung wird in einer Tabelle angezeigt, welche Abfragen erfolgreich waren (das XML-Fragment stimmte mit dem tatsächlichen Ergebnis überein) und welche nicht. Eine Möglichkeit, genauer zu erfahren, inwiefern die Abfrage fehlschlug, bestand nicht.

Diese Serienauswertungsfunktion wurde als erstem Schritt aus dem GUI-Code extrahiert (und ins Paket `preference.xpath.test` ausgelagert), so dass Tests unabhängig von der GUI automatisiert ausgeführt werden können. Eine Anbindung an das JUnit-Framework erlaubt es, diese Tests in JUnit-fähigen Entwicklungsumgebungen (wie beispielsweise Eclipse) automatisiert ausführen zu lassen und detaillierte Informationen über Erfolg oder Misserfolg der einzelnen Tests zu bekommen. Die GUI zeigt nun bei fehlgeschlagenen Tests auch an, welches Ergebnis sie lieferten und welches sie hätten liefern sollen. Zudem können die Testfälle jetzt auch über Apache Xindice ausgeführt werden.

Aus der Query-Favoritenliste lässt sich nun auch automatisch eine neue XML-Testcase-Datei erstellen. Die Namen der Testcases werden automatisch durchnummeriert; will man aussagekräftigere Namen, so muss man die Testcasedatei per Hand bearbeiten.

Wie bereits erwähnt waren die Testfälle nur XML-ähnlich – innerhalb der XPath-Anfragen wurden Zeichen mit spezieller Bedeutung wie `<`, `&` oder `>` nicht escaped. Aus diesem Grunde wurde die Testcasedatei auch nicht per XML-Parser ausgelesen, sondern es wurde mit Stringfunktionen nach den relevanten Tags gesucht. Diese Vorgehensweise ist zum einen fehleranfällig, zum anderen macht sie es schwierig, sich mit üblichen XML-Viewern einen Überblick über die Testcases zu verschaffen, weil die meisten XML-Viewer beim ersten Syntaxfehler zu parsen aufhören. Deshalb wurde das Parsen auf einen XML-Parser umgestellt und die existierenden Testfälle angepasst, so dass nun gültiges XML verwendet wird.

Ebenso wurden an allen Stellen, wo sich die Preference-XPath-Syntax geändert hat (im Wesentlichen bei Extensions, die jetzt fest eingebaut sind), natürlich auch die Testfälle mit angepasst, bzw. neue Funktionalitäten wie LAYERED-Präferenzen, Explicit-SV-Relation oder selbstdefinierte XPath-Funktionen erhielten neue Testcases. Außerdem wurden auch für behobene Fehler neue Testfälle angelegt, um ein Wiederauftauchen des Fehlers zu verhindern. Einer der Testfälle war fehlerhaft (das richtige Ergebnis wäre ein Fehler gewesen). Nachdem das XML-Format derartige Testfälle nicht erlaubt und der Testfall nun wirklich einen Fehler verursachen würde, wurde er entfernt.

Die Ergebnisse der meisten XPath-Anfragen sind Mengen von XML-Knoten. Das bisherige Testframework sah einen Test nur als erfolgreich an, wenn die Reihenfolge der Elemente in der Ergebnismenge übereinstimmte. Nachdem diese Reihenfolge aber aufgrund der internen Struktur der BMOSets nicht stabil ist (sie war es auch in der Ursprungsversion nicht), und auch in der Praxis bei verschiedenen Implementierungen verschiedene Ergebnisreihenfolgen auftreten können, wurde der Test dahingehend abgeschwächt, dass auch Permutationen der Elemente als Testerfolg gewertet werden. Auf der anderen Seite hat das alte Testframework die Werte von XML-Attributen ignoriert – es wurden also nur Elementknoten und Textknoten verglichen. Nachdem diese Teile in vielen XML-Datenbanken (die alle Daten in Attributen ablegen) aber bei jedem Datensatz identisch aussehen, waren Testfälle auf derartigen Datenbanken nicht sinnvoll einsetzbar. Deshalb werden Attributknoten jetzt mitvergleichen.

Zusätzlich zur Auswertung werden die XPath-Anfragen nun auch testweise geklont sowie nach XML serialisiert und deserialisiert. Dies hat einige Fehler in der Serialisierung und Deserialisierung von Präferenzen zutage gefördert. Das serialisierte XML wird zusätzlich gegen die DTD validiert. Des Weiteren wurde sichergestellt, dass die geklonte Abfrage `equal` zur ursprünglichen Anfrage ist und auch denselben HashCode erzeugt.

Weitere Testfälle, die nicht im XML-Format vorliegen, sondern als reine JUnit-Testcases, dienen auch dazu, die Bugfixes für gefundene Fehler zu testen und eine Regression zu verhindern. Dazu gehören beispielsweise ein Testfall, der zwei Queries überlappt ausführt und Probleme mit den ThreadLocals aufspüren half, und etliche Testfälle, die Parserfehler erkennen.

Der Rest der Test-GUI wurde vom Defaultpackage in ein eigenes Paket `preference.xpath.gui` verschoben.

Die Anbindung an das Preference-Framework erfolgt nun über die neue Fassadenklasse. Die Ergebnisknoten der Anfrage werden zu einem XML-Dokument zusammengebaut (vorher wurden sie einzeln serialisiert und dann die Strings zusammengesetzt), welches dann auf dem Bildschirm angezeigt wird. Diese Anzeige kann nun wahlweise als Text (wie vorher) oder als XML-Baum erscheinen. Der XML-Baum macht es einfach, uninteressante Teile des Ergebnisses zuzuklappen. Auch das QueryRewriting (das nun durch die Verwendung der neuen `QueryRewriting`-Klasse auch die Locpath-Arrays anzeigt) kann nun als Text oder XML-Baum angezeigt werden.

Die Test-GUI kann nun außer Xindice-Modus und Normal-Modus auch noch einen MultiDocument-Modus verwenden, bei dem Anfragen auf mehreren XML-Dateien simultan ausgeführt werden (wie es normalerweise nur bei Xindice geschieht). Des Weiteren können zusätzlich zu Score- und Rank-Funktionen auch benutzerdefinierte XPath-Funktionen zur Laufzeit registriert werden. Liegt eine zu registrierende Funktion nicht im Klassenpfad, so wird dieser automatisch erweitert.

Ansonsten wurde am GUI-Code nur das Nötigste getan. Er ist immer noch unübersichtlich und schwer wartbar. Nachdem die GUI aber nur zur internen Evaluation dient (alles brauchbare wurde ins Testframework oder in die Fassadenklasse ausgelagert), sollte dies zu verschmerzen sein.

B.4 Xindice-Anbindung

Xindice 1.0 („The name is pronounced zeen-dee-chay in your best faux Italian accent.“, [ASF05a]) ist ein XML-basiertes Open-Source-Datenbanksystem der Apache Foundation. Es dient dazu, XPath-Anfragen auf einer XML-Datenbank (bestehend aus Datensätzen, die ihrerseits kleine XML-Dokumente sind) auszuführen und die Ergebnisse zurückzugeben. Für Preference XPath existiert ein XPathResolver, der in Xindice eingebunden und dort für die Auswertung von Preference-XPath-Anfragen verwendet werden kann. Dieser Resolver war die einzige Anwendung für den MultiDocumentMode von Preference XPath. Für Xindice funktionierte der MultiDocumentMode ausreichend, weil Xindice reine XPath-Anfragen intern auswertet, ohne Preference XPath zu bemühen.

Aufgrund der Implementierung von Xindice und der vielen externen Abhängigkeiten ist es schwierig, Xindice auf anderen JDKs als 1.4.0 (für das es released wurde) zu compilieren. Unter JDK1.4.1 ist es compilier- und nutzbar, wenn man den mitgelieferten Object-Request-Broker OpenORB von 1.2.0 auf 1.2.1 (aber nicht weiter) aktualisiert (diese „historische“ Version von OpenORB findet sich in Sourceforges Downloadarchiv). Versuche, es auf anderen JDKs zum Laufen zu bringen, habe ich es eine Weile probiert und dann aufgegeben.

Aus diesem Grunde (und da die Xindice-Anbindung ohnehin nie wirklich genutzt wurde) habe ich die Anpassungen des Codes darauf beschränkt, den angepassten XPathResolver weiterhin unter JDK1.4.1 lauffähig zu halten. Der Xindice-spezifische Code in der Beispiel-GUI wurde in eigene Klassen ausgelagert um die GUI auch ohne Xindice-Bibliotheken benutzen zu können.

B.5 API-Dokumentation

Die API-Dokumentation der Preference-XPath-Klassen wurde an die tatsächlichen Verhältnisse angepasst. Die bisherige Dokumentation war gemischt deutsch und englisch; alles neu hinzugekommene wurde auf Englisch dokumentiert. Ebenso wurde das Preference-XPath-Tutorial an die neuen Begebenheiten angepasst.

Anhang C

Rohdaten der Performance-Analysen

Die Rohdaten aller Diagramme befinden sich auf der beigefügten CD.

Die folgende Tabelle listet die genauen XPath-Anfragen auf:

Name	Anfrage	Datenbank
DUMMY1	/db/record#[name/@value highest]#	DUMMY ⁺ ₁₀₀ (1000)
DUMMY2	/db/record#[@value highest]#	DUMMY ⁺ ₁₀₀ (1000)
250AUTO	/AUTOS/MODELL#[MOTOR/VERBRAUCH highest]#	AUTOS (250x)
500Pref	/db/elem[@value='42']#[@value highest]# #[@value is '0']#	SMALL (500x)
500NoPref	/db/elem[@value='42']#[@value highest]# #[@value is '0']#	SMALL (500x)
COSIMA-X	/BMECAT/T_NEW_CATALOG/ARTICLE	COSIMA
COSIMA-Y	/BMECAT/T_NEW_CATALOG/ARTICLE #[SUPPLIER_AID is '4711']#	COSIMA (alle)
COSIMA-A	/BMECAT/T_NEW_CATALOG/ARTICLE #[SUPPLIER_AID isnt '95590']#	COSIMA
COSIMA-B	/BMECAT/T_NEW_CATALOG/ARTICLE #[SUPPLIER_AID is '1134']#	COSIMA (1.DS)

Hier folgen noch einige Diagramme, die aus den Rohdaten generiert wurden:

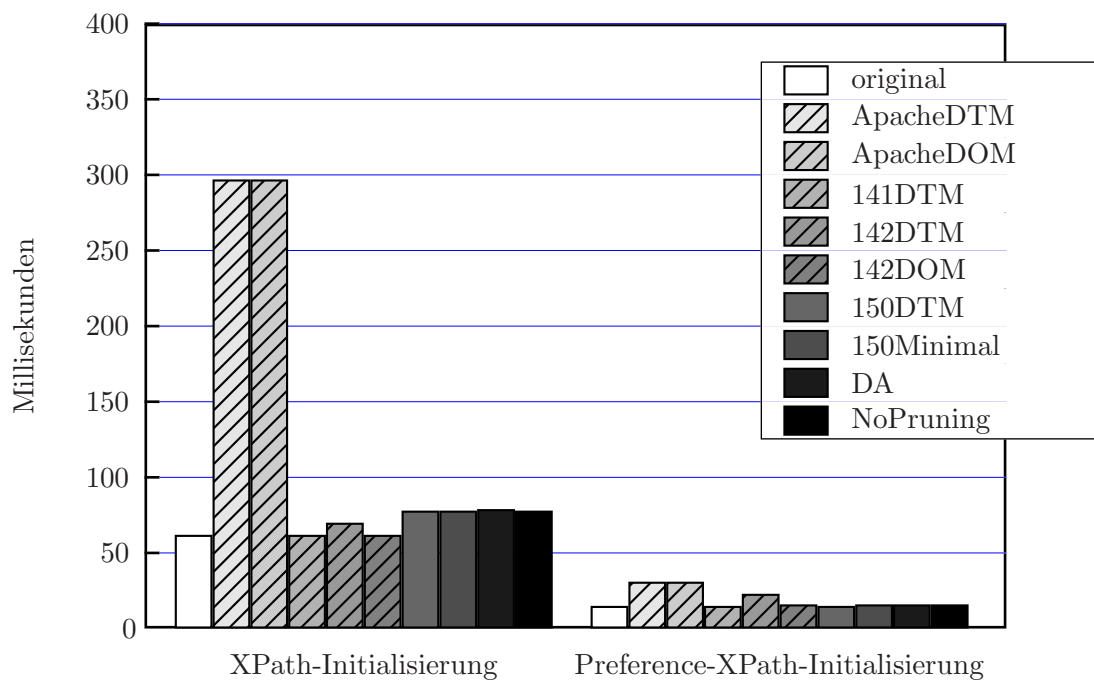


Abbildung C.1: Initialisierungszeiten im Vergleich

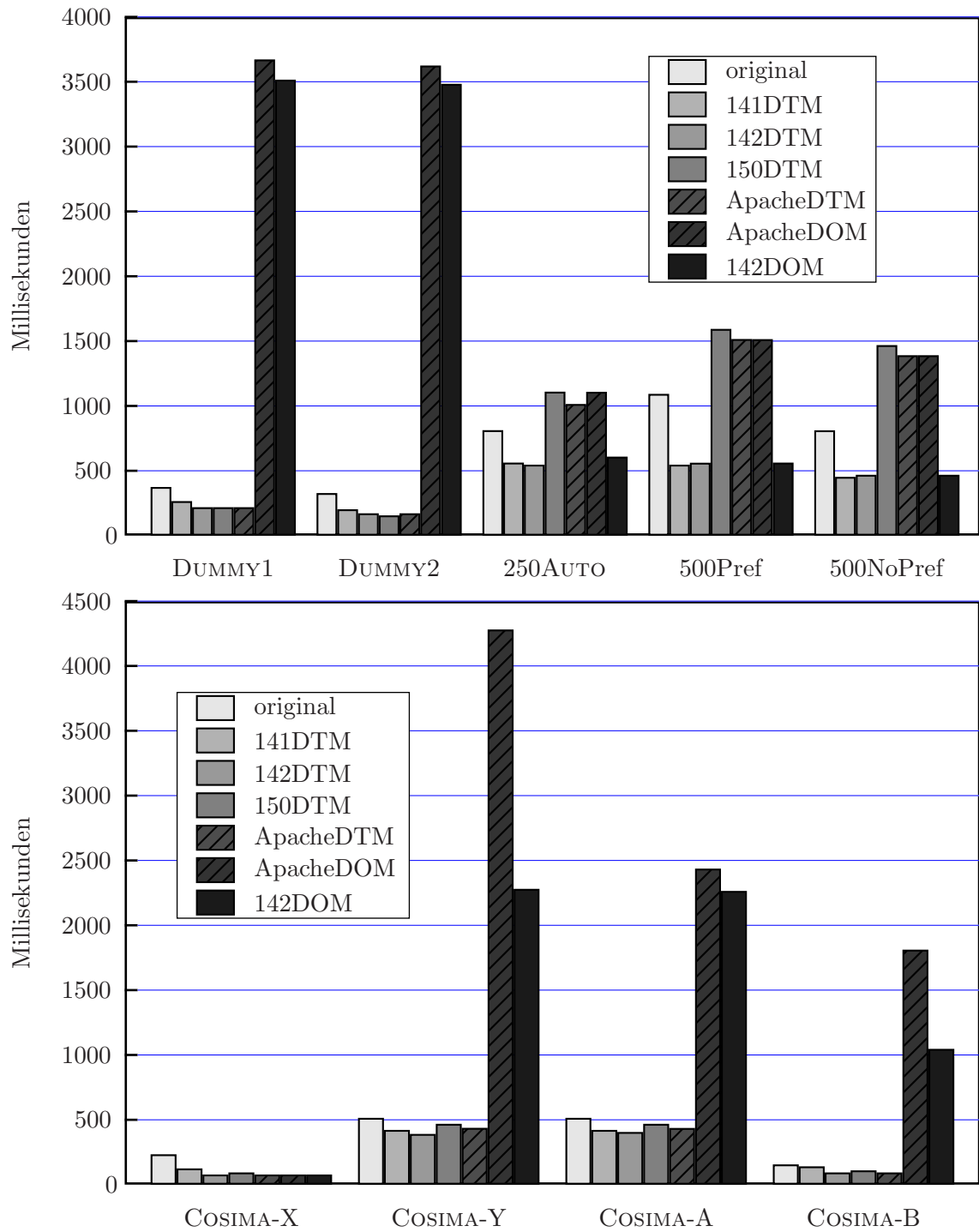


Abbildung C.2: Evaluation der DOM/DTM-Implementationen

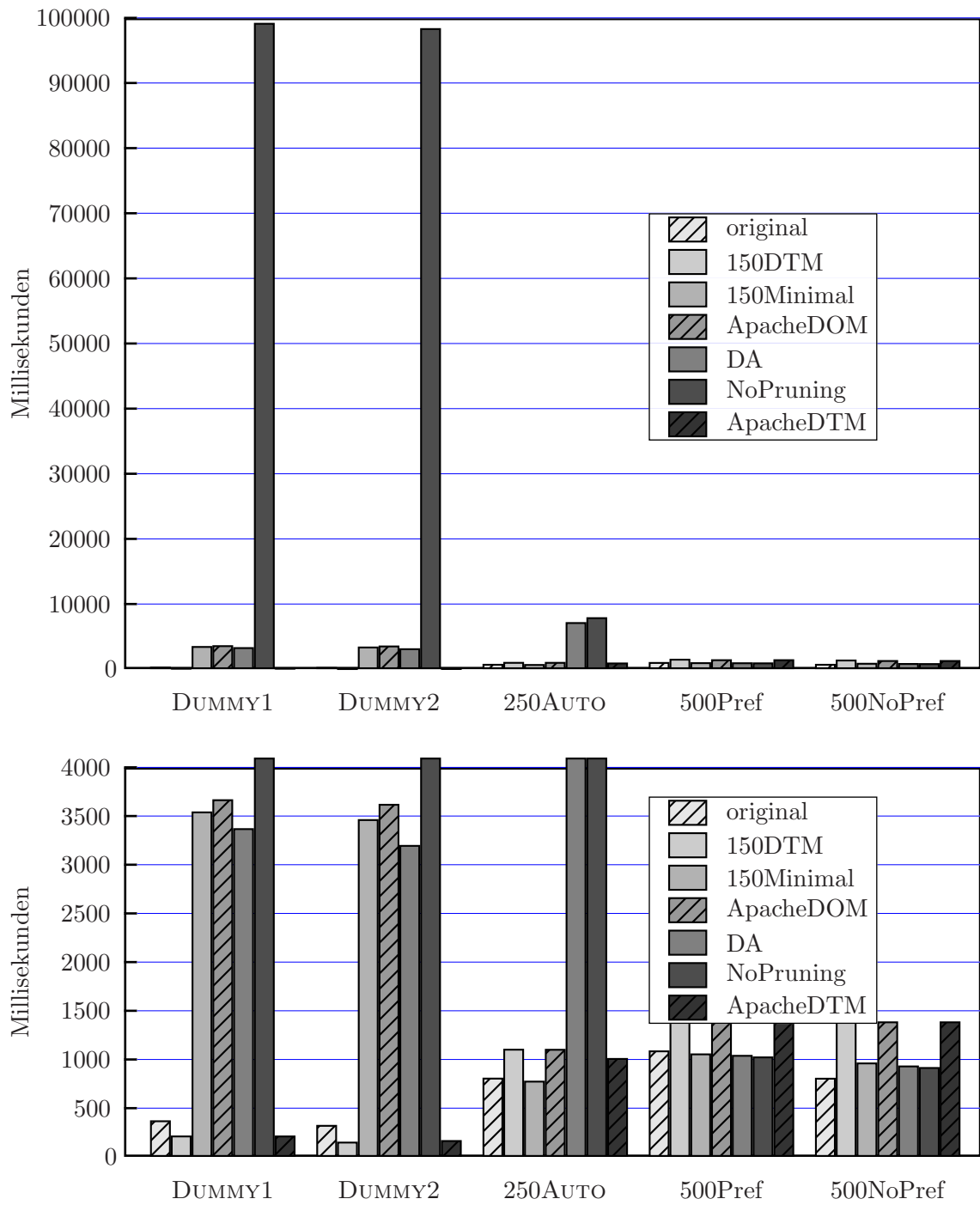


Abbildung C.3: Evaluation der JDK1.5-Implementationen, Teil 1, unten abgeschnitten

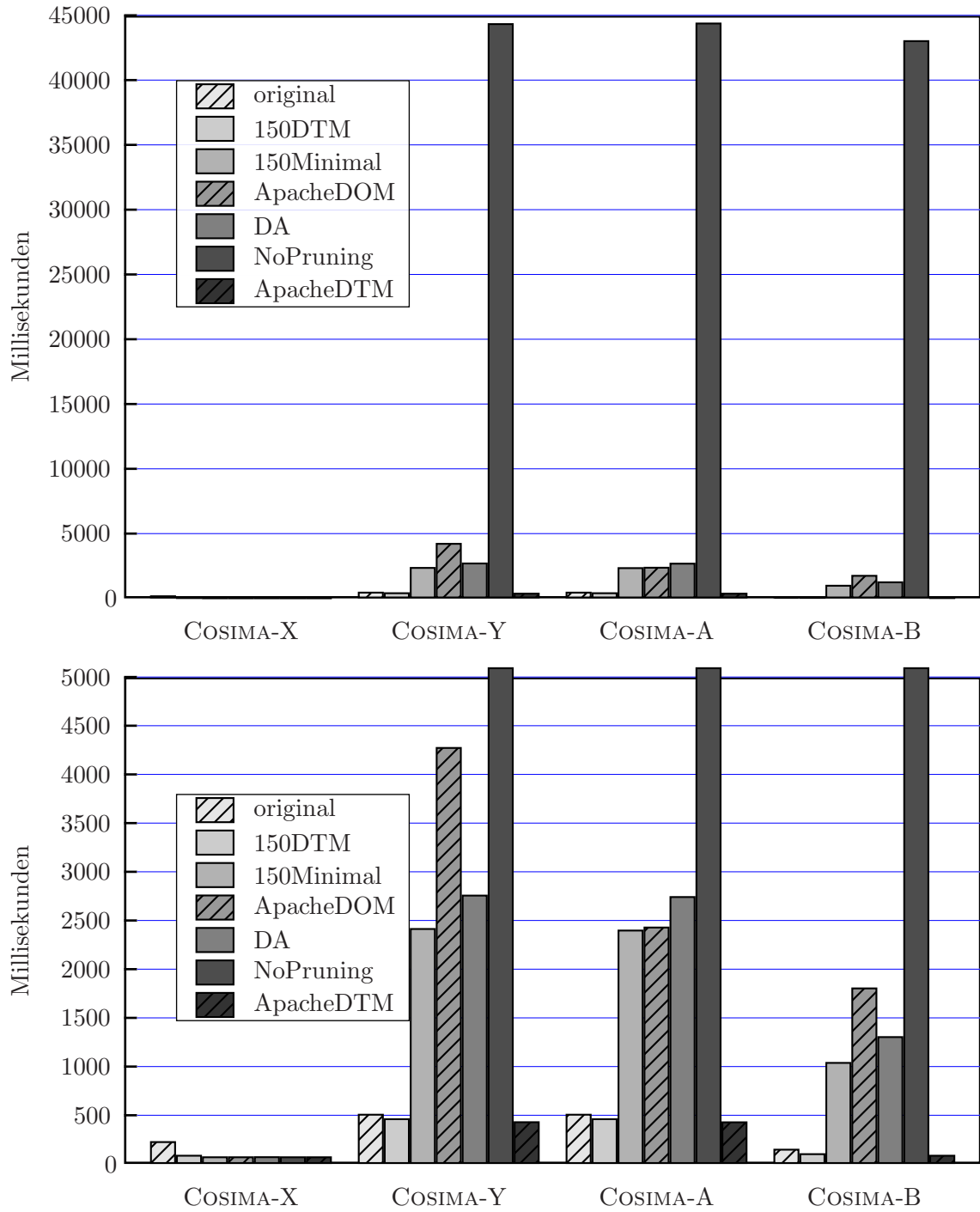


Abbildung C.4: Evaluation der JDK1.5-Implementationen, Teil 2, unten abgeschnitten